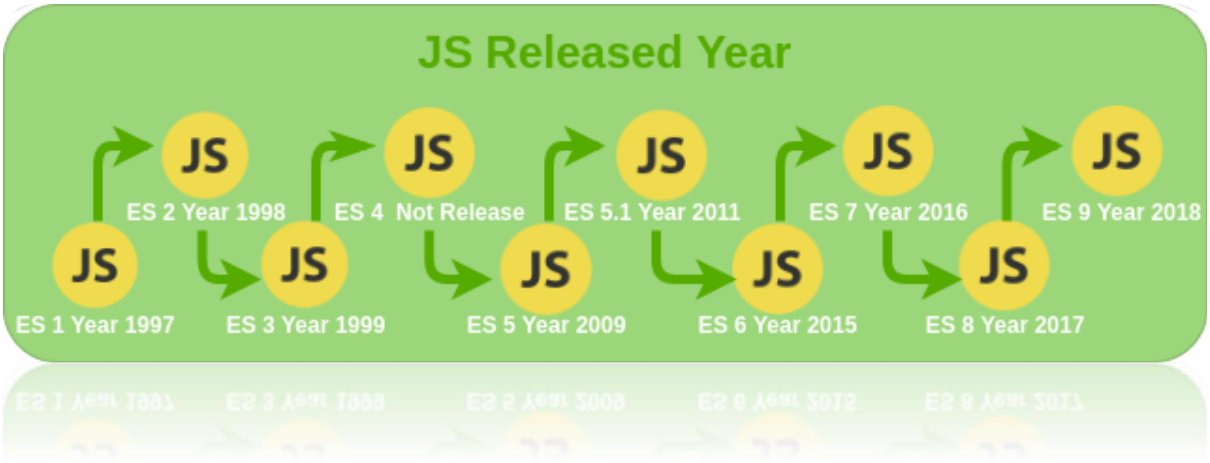
History of JavaScript: It was created in 1995 by **Brendan Eich** while he was an engineer at Netscape. It was originally going to be named LiveScript but was renamed. Unlike most programming languages, the JavaScript language has no concept of input or output. It is designed to run as a scripting language in a host environment, and it is up to the host environment to provide mechanisms for communicating with the outside world. The most common host environment is the browser.

JavaScript is an interpreted programming language that conforms to the ECMAScript specification. JavaScript is high-level, often just-in-time compiled, and multi-paradigm. It has curly-bracket syntax, dynamic typing, prototype-based object-orientation, and first-class functions. In this path you will learn the basics of JavaScript as well as more advanced topics such as promises, asynchronous programming, proxies and reflection.

Java Script Overview & Basics

JavaScript is the dominant client-side scripting language of the Web, with 98% of all websites (mid–2022) using it for this purpose. Scripts are embedded in or included from HTML documents and interact with the DOM. All major web browsers have a built-in JavaScript engine that executes the code on the user's device.

- **Client-side**: It supplies objects to control a browser and its Document Object Model (DOM). Like if client-side extensions allow an application to place elements on an HTML form and respond to user events such as mouse clicks, form input, and page navigation. Useful libraries for the client-side are AngularJS, ReactJS, VueJS and so many others.
- **Server-side**: It supplies objects relevant to running JavaScript on a server. Like if the server-side extensions allow an application to communicate with a database, and provide continuity of information from one invocation to another of the application, or perform file manipulations on a server. The useful framework which is the most famous these days is node.js.
- **Imperative language** – In this type of language we are mostly concern about how it is to be done. It simply controls the flow of computation. The procedural programming approach, object, oriented approach comes under this like async await we are thinking what it is to be done further after async call.
- **Declarative programming** – In this type of language we are concern about how it is to be done, basically here logical computation requires. Here main goal is to describe the desired result without direct dictation on how to get it like arrow function do.



JavaScript can be added to your HTML file in two ways:

Internal JS: We can add JavaScript directly to our HTML file by writing the code inside the <script> tag. The <script> tag can either be placed inside the <head> or the <body> tag according to the requirement.

Syntax:

```
<script>
// JavaScript Code
</script>
```

External JS: We can write JavaScript code in other file having an extension.js and then link this file inside the <head> tag of the HTML file in which we want to add this code.

```
<!DOCTYPE html>
<html>
<body>
        <h2>External JavaScript</h2>
        <p id="demo">Geetanjali College</p>
        <button type="button" onclick="myFunction()">Try it</button>
<script src="myScript.js"></script>
</body>
</html>
```

Advantages of External JavaScript:

- **Cached JavaScript files can speed up page loading**
- **It makes JavaScript and HTML easier to read and maintain**
- **It separates the HTML and JavaScript code**
- **It focuses on code re usability that is one JavaScript Code can run in various HTML files.**

JavaScript in body or head: Scripts can be placed inside the body or the head section of an HTML page or inside both head and body.

JavaScript in head: A JavaScript function is placed inside the head section of an HTML page and the function is invoked when a button is clicked.

JavaScript in body: A JavaScript function is placed inside the body section of an HTML page and the function is invoked when a button is clicked.

Variable, Conditional Statements, Loops in JS

A **JavaScript variable** is simply a name of storage location. There are two types of variables in JavaScript: **local** variable and **global** variable.
There are some rules while declaring a JavaScript variable (also known as identifiers).

1. Name must start with a letter (a to z or A to Z), underscore (_), or dollar($) sign.
2. After first letter we can use digits (0 to 9), for example value1.
3. JavaScript variables are case sensitive, for example x and X are different variables.

Example of JavaScript variable

```
<script>
var x = 10;
var y = 20;
var z=x+y;
document.write(z);
</script>
```

Output

30

JavaScript local variable

A JavaScript local variable is declared inside block or function. It is accessible within the function or block only.
For example:

```
<script>
function abc(){
        var x=10;//local variable
}
</script>
```

JavaScript global variable

A JavaScript global variable is accessible from any function. A variable i.e. declared outside the function or declared with window object is known as global variable. For example:

```
<script>
var data=200;//gloabal variable
function a(){
        document.writeln(data);
}
function b(){
        document.writeln(data);
}
a();//calling JavaScript function
b();
</script>
```

Output

200

JavaScript If-else

The JavaScript if-else statement is used to execute the code whether condition is true or false. There are three forms of if statement in JavaScript.

- If Statement
- If else statement
- if else if statement

JavaScript If statement

It evaluates the content only if expression is true. The signature of JavaScript if statement is given below.

```
if(expression){
//content to be evaluated
}
<script>
var a=20;
if(a>10){
        document.write("value of a is greater than 10");
}
</script>
```

Output

value of a is greater than 10

JavaScript If...else Statement

It evaluates the content whether condition is true of false. The syntax of JavaScript if-else statement is given below.

```
if(expression){
        //content to be evaluated if condition is true
}
else{
        //content to be evaluated if condition is false
}
<script>
var a=20;
if(a%2==0){
        document.write("a is even number");
}
else{
        document.write("a is odd number");
}
</script>
```

Output

a is even number

JavaScript If...else if statement

It evaluates the content only if expression is true from several expressions. The signature of JavaScript if else if statement is given below.

```
if(expression1){
//content to be evaluated if expression1 is true
}
```

```
else if(expression2){
//content to be evaluated if expression2 is true
}
else if(expression3){
//content to be evaluated if expression3 is true
}
else{
//content to be evaluated if no expression is true
}
```

```
<script>
var a=20;
if(a==10){
        document.write("a is equal to 10");
}
else if(a==15){
        document.write("a is equal to 15");
}
else if(a==20){
        document.write("a is equal to 20");
}
else{
        document.write("a is not equal to 10, 15 or 20");
}
</script>
```

Output

a is equal to 20

JavaScript Switch

The JavaScript switch statement is used to execute one code from multiple expressions. It is just like else if statement that we have learned in previous page. But it is convenient than if..else..if because it can be used with numbers, characters etc.

The signature of JavaScript switch statement is given below.

```
switch(expression){
        case value1:
                code to be executed;
        break;
        case value2:
                code to be executed;
        break;
        ......

        default:
                code to be executed if above values are not matched;
}
```

```
<script>
var grade='B';
var result;
switch(grade){
	case 'A':
		result="A Grade";
	break;
	case 'B':
		result="B Grade";
	break;
	case 'C':
		result="C Grade";
	break;
	default:
		result="No Grade";
}
document.write(result);
</script>
```

JavaScript Loops

The JavaScript loops are used to iterate the piece of code using for, while, do while or for-in loops. It makes the code compact. It is mostly used in array.

There are four types of loops in JavaScript.

- **for loop**
- **while loop**
- **do-while loop**
- **for-in loop**

JavaScript For loop

The JavaScript for loop iterates the elements for the fixed number of times. It should be used if number of iteration is known. The syntax of for loop is given below.

```
for (initialization; condition; increment){
	code to be executed
}
```

Let's see the simple example of for loop in javascript.

```
<script>
for (i=1; i<=5; i++)
{
	document.write(i + "<br/>")
}
</script>
```

JavaScript while loop

The JavaScript while loop iterates the elements for the infinite number of times. It should be used if number of iteration is not known. The syntax of while loop is given below.

```
while (condition){
        code to be executed
}
```

Let's see the simple example of while loop in javascript.

```
<script>
var i=11;
while (i<=15){
        document.write(i + "<br/>");
        i++;
}
</script>
```

JavaScript do while loop

The JavaScript do while loop iterates the elements for the infinite number of times like while loop. But, code is executed at least once whether condition is true or false. The syntax of do while loop is given below.

```
do{
        code to be executed
}while (condition);
```

Let's see the simple example of do while loop in javascript.

```
<script>
var i=21;
do{
        document.write(i + "<br/>");
        i++;
}while (i<=25);
</script>
```

Functions, Arrays & Events in JS

JavaScript Functions

JavaScript functions are used to perform operations. We can call JavaScript function many times to reuse the code.

Advantage of JavaScript function

There are mainly two advantages of JavaScript functions.

Code reusability: We can call a function several times so it save coding.

Less coding: It makes our program compact. We don't need to write many lines of code each time to perform a common task.

JavaScript Function Syntax

The syntax of declaring function is given below.

```
function functionName([arg1, arg2, ...argN]){
//code to be executed
}
```

JavaScript Functions can have 0 or more arguments.

JavaScript Function Example

Let's see the simple example of function in JavaScript that does not has arguments.

```
<script>
function msg(){
        alert("hello! this is message");
}
</script>
<input type="button" onclick="msg()" value="call function"/>
```

JavaScript Function Arguments

We can call function by passing arguments. Let's see the example of function that has one argument.

```
<script>
function getcube(number){
        alert(number*number*number);
}
</script>
<form>
        <input type="button" value="click" onclick="getcube(4)"/>
</form>
```

Function with Return Value

We can call function that returns a value and use it in our program. Let's see the example of function that returns value.

```
<script>
function getInfo(){
        return "hello hardikchavda! How r u?";
}
</script>
<script>
        document.write(getInfo());
</script>
```

JavaScript Function Object

In JavaScript, the purpose of Function constructor is to create a new Function object. It executes the code globally. However, if we call the constructor directly, a function is created dynamically but in an unsecured way.

Syntax

```
new Function ([arg1[, arg2[, ....argn]],] functionBody)
```

Parameter

- arg1, arg2, ... , argn - It represents the argument used by function.
- functionBody - It represents the function definition.
-

Let's see function methods with description.

Method	Description
apply()	It is used to call a function contains this value and a single array of arguments.
bind()	It is used to create a new function.
call()	It is used to call a function contains this value and an argument list.
toString()	It returns the result in a form of a string.

JavaScript Function Object Examples

Example 1

Let's see an example to display the sum of given numbers.

```
<script>
var add=new Function("num1","num2","return num1+num2");
document.writeln(add(2,5));
</script>
```

Example 2

Let's see an example to display the power of provided value.

```
<script>
var pow=new Function("num1","num2","return Math.pow(num1,num2)");
document.writeln(pow(2,3));
</script>
```

JavaScript Array

JavaScript array is an object that represents a collection of similar type of elements.

There are 3 ways to construct array in JavaScript

By array literal
By creating instance of Array directly (using new keyword)
By using an Array constructor (using new keyword)

JavaScript array literal

The syntax of creating array using array literal is given below:

```
var arrayname=[value1,value2.....valueN];
```

As you can see, values are contained inside [ ] and separated by , (comma).

Let's see the simple example of creating and using array in JavaScript.

```
<script>
var emp=["Sonoo","Vimal","Ratan"];
for (i=0;i<emp.length;i++){
        document.write(emp[i] + "<br/>");
}
</script>
//The .length property returns the length of an array.
```

Output

```
Sonoo
Vimal
Ratan
```

JavaScript Array directly (new keyword)

The syntax of creating array directly is given below:

```
var arrayname=new Array();
```

Here, new keyword is used to create instance of array.

Let's see the example of creating array directly.

```
<script>
var i;
var emp = new Array();
emp[0] = "Arun";
emp[1] = "Varun";
emp[2] = "John";

for (i=0;i<emp.length;i++){
        document.write(emp[i] + "<br>");
}
</script>
```

Output

```
Arun
Varun
John
```

JavaScript array constructor (new keyword)

Here, you need to create instance of array by passing arguments in constructor so that we don't have to provide value explicitly.

The example of creating object by array constructor is given below.

```
<script>
var emp=new Array("Jai","Vijay","Smith");
for (i=0;i<emp.length;i++){
        document.write(emp[i] + "<br>");
}
</script>
```

Output

Jai
Vijay
Smith

JavaScript Array Methods

Let's see the list of JavaScript array methods with their description.

Methods	Description
concat()	It returns a new array object that contains two or more merged arrays.
copywithin()	It copies the part of the given array with its own elements and returns the modified array.
entries()	It creates an iterator object and a loop that iterates over each key/value pair.
every()	It determines whether all the elements of an array are satisfying the provided function conditions.
flat()	It creates a new array carrying sub-array elements concatenated recursively till the specified depth.
flatMap()	It maps all array elements via mapping function, then flattens the result into a new array.
fill()	It fills elements into an array with static values.
from()	It creates a new array carrying the exact copy of another array element.
filter()	It returns the new array containing the elements that pass the provided function conditions.
find()	It returns the value of the first element in the given array that satisfies the specified condition.
findIndex()	It returns the index value of the first element in the given array that satisfies the specified condition.
forEach()	It invokes the provided function once for each element of an array.
includes()	It checks whether the given array contains the specified element.
indexOf()	It searches the specified element in the given array and returns the index

	of the first match.
isArray()	It tests if the passed value ia an array.
join()	It joins the elements of an array as a string.
keys()	It creates an iterator object that contains only the keys of the array, then loops through these keys.
lastIndexOf()	It searches the specified element in the given array and returns the index of the last match.
map()	It calls the specified function for every array element and returns the new array
of()	It creates a new array from a variable number of arguments, holding any type of argument.
pop()	It removes and returns the last element of an array.
push()	It adds one or more elements to the end of an array.
reverse()	It reverses the elements of given array.
reduce(function, initial)	It executes a provided function for each value from left to right and reduces the array to a single value.
reduceRight()	It executes a provided function for each value from right to left and reduces the array to a single value.
some()	It determines if any element of the array passes the test of the implemented function.
shift()	It removes and returns the first element of an array.
slice()	It returns a new array containing the copy of the part of the given array.
sort()	It returns the element of the given array in a sorted order.
splice()	It add/remove elements to/from the given array.
toLocaleString()	It returns a string containing all the elements of a specified array.
toString()	It converts the elements of a specified array into string form, without affecting the original array.
unshift()	It adds one or more elements in the beginning of the given array.
values()	It creates a new iterator object carrying values for each index in the array.

JavaScript Events

The change in the state of an object is known as an Event. In html, there are various events which represents that some activity is performed by the user or by the browser. When javascript code is included in HTML, js react over these events and allow the execution. This process of reacting over the events is called Event Handling. Thus, js handles the HTML events via Event Handlers.

For example, when a user clicks over the browser, add js code, which will execute the task to be performed on the event.

Some of the HTML events and their event handlers are:

Mouse events:

Event Performed	Event Handler	Description
click	onclick	When mouse click on an element
mouseover	onmouseover	When the cursor of the mouse comes over the element
mouseout	onmouseout	When the cursor of the mouse leaves an element
mousedown	onmousedown	When the mouse button is pressed over the element
mouseup	onmouseup	When the mouse button is released over the element
mousemove	onmousemove	When the mouse movement takes place.

Keyboard events:

Event Performed	Event Handler	Description
Keydown & Keyup	onkeydown & onkeyup	When the user press and then release the key

Form events:

Event Performed	Event Handler	Description
focus	onfocus	When the user focuses on an element
submit	onsubmit	When the user submits the form
blur	onblur	When the focus is away from a form element
change	onchange	When the user modifies or changes the value of a form element

Window/Document events

Event Performed	Event Handler	Description
load	onload	When the browser finishes the loading of the page
unload	onunload	When the visitor leaves the current webpage, the browser unloads it
resize	onresize	When the visitor resizes the window of the browser

Click Event

```
<html>
<head> Javascript Events </head>
<body>
<script language="Javascript" type="text/Javascript">
function clickevent()
{
        document.write("This is HardikChavda");
}
</script>
        <form>
                <input type="button" onclick="clickevent()" value="Who's this?"/>
        </form>
</body>
</html>
```

MouseOver Event

```
<html>
<head>
<h1> Javascript Events </h1>
</head>
<body>
<script language="Javascript" type="text/Javascript">
function mouseoverevent()
{
        alert("This is HardikChavda");
}
</script>
        <p onmouseover="mouseoverevent()"> Keep cursor over me</p>
</body>
</html>
```

Focus Event

```
<html>
<head> Javascript Events</head>
<body>
        <h2> Enter something here</h2>
        <input type="text" id="input1" onfocus="focusevent()"/>
<script>
function focusevent()
{
        document.getElementById("input1").style.background=" aqua";
}
</script>
</body>
</html>
```

Keydown Event

```
<html>
<head> Javascript Events</head>
<body>
<h2> Enter something here</h2>
<input type="text" id="input1" onkeydown="keydownevent()"/>
<script>
function keydownevent()
{
document.getElementById("input1");
alert("Pressed a key");
}
</script>
</body>
</html>
Load event
<html>
<head>Javascript Events</head>
</br>
<body onload="window.alert('Page successfully loaded');">
<script>
document.write("The page is loaded successfully");
</script>
</body>
</html>
```

ES6 Overview & Basics

ES6 or ECMAScript 6 is a scripting language specification which is standardized by ECMAScript International. This specification governs some languages such as JavaScript, ActionScript, and Jscript. ECMAScript is generally used for client-side scripting, and it is also used for writing server applications and services by using Node.js.

ES6 allows you to write the code in such a way that makes your code more modern and readable. By using ES6 features, we write less and do more, so the term 'Write less, do more' suits ES6.

What is ES6?

ES6 is an acronym of ECMAScript 6 and also known as ECMAScript 2015.

ES6 or ECMAScript6 is a scripting language specification which is standardized by ECMAScript International. It is used by the applications to enable client-side scripting. This specification is affected by programming languages like Self, Perl, Python, Java, etc. This specification governs some languages such as JavaScript, ActionScript, and Jscript. ECMAScript is generally used for client-side scripting, and it is also used for writing server applications and services by using Node.js.

ES6 allows you to make the code more modern and readable. By using ES6 features, we write less and do more, so the term 'Write less, do more' suits ES6. ES6 introduces you many great features such as scope variable, arrow functions, template strings, class destructions, modules, etc.

ES6 was created to standardize JavaScript to help several independent implementations. Since the standard was first published, JavaScript has remained the well-known implementation of ECMAScript, comparison to other most famous implementations such as Jscript and ActionScript.

History

The ECMAScript specification is the standardized specification of scripting language, which is developed by Brendan Eich (He is an American technologist and the creator of JavaScript programming language) of Netscape (It is a name of brand which is associated with Netscape web browser's development).

Initially, the ECMAScript was named Mocha, later LiveScript, and finally, JavaScript. In December 1995, Sun Microsystems (an American company that sold the computers and its components, software, and IT services. It created Java, NFS, ZFS, SPARC, etc.) and Netscape announced the JavaScript during a press release.

During November 1996, Netscape announced a meeting of the ECMA International standard organization to enhance the standardization of JavaScript.

ECMA General Assembly adopted the first edition of ECMA-262 in June 1997. Since then, there are several editions of the language standard have published. The Name 'ECMAScript' was a settlement between the organizations which included the standardizing of the language, especially Netscape and Microsoft, whose disputes dominated the primary standard sessions. Brendan Eich commented that 'ECMAScript was always an unwanted trade name which sounds like a skin disease (eczema).'

Both JavaScript and Jscript aim to be compatible with the ECMAScript, and they also provide some of the additional features that are not described in ECMA specification.

Prerequisite

Before learning ES6, you should have a basic understanding of JavaScript.

ES6 Classes, functions & Promises

ES6 Classes

Classes are an essential part of object-oriented programming (OOP). Classes are used to define the blueprint for real-world object modeling and organize the code into reusable and logical parts.

Before ES6, it was hard to create a class in JavaScript. But in ES6, we can create the class by using the class keyword. We can include classes in our code either by class expression or by using a class declaration.

A class definition can only include constructors and functions. These components are together called as the data members of a class. The classes contain constructors that allocates the memory to the objects of a class. Classes contain functions that are responsible for performing the actions to the objects.

Syntax: Class Expression

```
var var_name = new class_name {}
```

Syntax: Class Declaration

```
class Class_name{}
```

Let us see the illustration for the class expression and class declaration.

Example - Class Declaration

```
class Student{
        constructor(name, age){
        this.name = name;
        this.age = age;
        }
}
```

Example - Class Expression

```
var Student = class{
        constructor(name, age){
        this.name = name;
        this.age = age;
        }
}
```

Instantiating an Object from class

Like other object-oriented programming languages, we can instantiate an object from class by using the new keyword.

Syntax

```
var obj_name = new class_name([arguements])
```

Accessing functions

The object can access the attributes and functions of a class. We use the '.' dot notation (or period) for accessing the data members of the class.

Syntax

```
obj.function_name();
```

Example

```
'use strict'
class Student {
        constructor(name, age) {
        this.n = name;
        this.a = age;
}
stu() {
        console.log("The Name of the student is: ", this.n)
        console.log("The Age of the student is: ",this. a)
      }
}
var stuObj = new Student('Peter',20);
stuObj.stu();
//The function stu() in the class will print the values of name and age.
```

Output

```
The Name of the student is:  Peter
The Age of the student is:  20
```

In the above example, we have declared a class Student. The constructor of the class contains two arguments name and age, respectively. The keyword 'this' refers to the current instance of the class. We can also say that the above constructor initializes two variables 'n' and 'a' along with the parameter values passed to the constructor.

Note: Including a constructor definition is mandatory in class because, by default, every class has a constructor.

The Static keyword

The static keyword is used for making the static functions in the class. Static functions are referenced only by using the class name.

Example

```
'use strict'
class Example {
        static show() {
        console.log("Static Function")
        }
}
Example.show() //invoke the static method
```

Class inheritance

Before the ES6, the implementation of inheritance required several steps. But ES6 simplified the implementation of inheritance by using the extends and super keyword.

Inheritance is the ability to create new entities from an existing one. The class that is extended for creating newer classes is referred to as superclass/parent class, while the newly created classes are called subclass/child class.

A class can be inherited from another class by using the 'extends' keyword. Except for the constructors from the parent class, child class inherits all properties and methods.

Syntax

```
class child_class_name extends parent_class_name{}
```

A class inherits from the other class by using the extends keyword.

Example

```
'use strict'
class Student {
        constructor(a) {
        this.name = a;
        }
}
class User extends Student {
        show() {
        console.log("The name of the student is:  "+this.name)
        }
}
var obj = new User('Sahil');
obj.show()
```

Output

```
The name of the student is:  Sahil
```

In the above example, we have declared a class student. By using the extends keyword, we can create a new class User that shares the same characteristics as its parent class Student. So, we can see that there is an inheritance relationship between these classes.

Types of inheritance

Inheritance can be categorized as Single-level inheritance, Multiple inheritance, and Multi-level inheritance. Multiple inheritance is not supported in ES6.

Single-level Inheritance

It is defined as the inheritance in which a derived class can only be inherited from only one base class. It allows a derived class to inherit the behavior and properties of a base class, which enables the reusability of code as well as adding the new features to the existing code. It makes the code less repetitive.

Multiple Inheritance

In multiple inheritance, a class can be inherited from several classes. It is not supported in ES6.

Multi-level Inheritance

In Multi-level inheritance, a derived class is created from another derived class. Thus, a multi-level inheritance has more than one parent class.

Let us understand it with the following example.

```
class Animal{
        eat(){
                console.log("eating...");
        }
}
class Dog extends Animal{
        bark(){
                console.log("barking...");
        }
}
class BabyDog extends Dog{
        weep(){
                console.log("weeping...");
        }
}
var d=new BabyDog();
d.eat(); d.bark(); d.weep();
```

Output

```
eating...
barking...
weeping...
```

Method Overriding and Inheritance

It is a feature that allows a child class to provide a specific implementation of a method which has been already provided by one of its parent class.

There are some rules defined for method overriding -

- **The method name must be the same as in the parent class.**
- **Method signatures must be the same as in the parent class.**

Let us try to understand the illustration for the same:

Example

```
'use strict' ;
class Parent {
        show() {
                console.log("It is the show() method from the parent class");
        }
}
class Child extends Parent {
        show() {
                console.log("It is the show() method from the child class");
        }
}
var obj = new Child();
obj.show();
```

Output

It is the show() method from the child class

In the above example, the implementation of the superclass function has changed in the child class.

The super keyword

It allows the child class to invoke the properties, methods, and constructors of the immediate parent class. It is introduced in ECMAScript 2015 or ES6. The super.prop and super[expr] expressions are readable in the definition of any method in both object literals and classes.

Syntax

```
super(arguments);
```

In this example, the characteristics of the parent class have been extended to its child class. Both classes have their unique properties. Here, we are using the super keyword to access the property from parent class to the child class.

```
'use strict' ;
class Parent {
        show() {
```

```
                console.log("It is the show() method from the parent class");
        }
}
class Child extends Parent {
        show() {
                super.show();
        console.log("It is the show() method from the child class");
        }
}
var obj = new Child();
obj.show();
```

Output

```
It is the show() method from the parent class
It is the show() method from the child class
```

ES6 Functions

A function is the set of input statements, which performs specific computations and produces output. It is a block of code that is designed for performing a particular task. It is executed when it gets invoked (or called). Functions allow us to reuse and write the code in an organized way. Functions in JavaScript are used to perform operations.

In JavaScript, functions are defined by using function keyword followed by a name and parentheses (). The function name may include digits, letters, dollar sign, and underscore. The brackets in the function name may consist of the name of parameters separated by commas. The body of the function should be placed within curly braces {}.

The syntax for defining a standard function is as follows:

```
function function_name() {
//body of the function
}
```

To force the execution of the function, we must have to invoke (or call) the function. It is known as function invocation. The syntax for invoking a function is as follows:

```
function_name()
```

Example

```
function hello() //defining a function
{
console.log ("hello function called");
}
hello(); //calling of function
```

Output

```
hello function called
```

In the above code, we have defined a function hello(). The pair of parentheses {} define the body of the function, which is called a scope of function.

Let us try to understand different functions.

Parameterized functions

Parameters are the names that are listed in the definition of the function. They are the mechanism of passing the values to functions.

The values of the parameters are passed to the function during invocation. Unless it is specified explicitly, the number of values passed to a function must match with the defined number of parameters.

Syntax

```
function function_name( parameter1,parameter2 ,…..parameterN) {
//body of the function
}
```

Example

In this example of parameterized function, we are defining a function mul(), which accepts two parameters x and y, and it returns their multiplication in the result. The parameter values are passed to the function during invocation.

```
function mul( x , y) {
        var c = x * y;
        console.log("x = "+x);
        console.log("y = "+y);
        console.log("x * y = "+c);
}
mul(20,30);
```

Output

```
x = 20
y = 30
x * y = 600
```


Default Function Parameters

In ES6, the function allows the initialization of parameters with default values if the parameter is undefined or no value is passed to it.

You can see the illustration for the same in the following code:

For example

```
function show (num1, num2=200)
{
        console.log("num1 = " +num1);
        console.log("num2 = " +num2);
}
show(100);
```

Output

```
100 200
```

In the above function, the value of num2 is set to 200 by default. The function will always consider 200 as the value of num2 if no value of num2 is explicitly passed.

The default value of the parameter 'num2' will get overwritten if the function passes its value explicitly. You can understand it by using the following example:

For example

```
function show(num1, num2=200)
{
        console.log("num1 = " +num1);
        console.log("num2 = " +num2);
}
show(100,500);
```

Output

100 500

Rest Parameters

Rest parameters do not restrict you to pass the number of values in a function, but all the passed values must be of the same type. We can also say that rest parameters act as the placeholders for multiple arguments of the same type.

For declaring the rest parameter, the name of the parameter should be prefixed with the spread operator that has three periods (not more than three or not less than three).

You can see the illustration for the same in the following example:

```
function show(a, ...args)
{
        console.log(a + " " + args);
}
show(50,60,70,80,90,100);
```

Output

50,60,70,80,90,100

Note: The rest parameters should be at last in the list of function parameters.

Returning functions

The function also returns the value to the caller by using the return statement. These functions are known as Returning functions. A returning function should always end with a return statement. There can be only one return statement in a function, and the return statement should be the last statement in the function.

When JavaScript reaches the return statement, the function stops the execution and exits immediately. That's why you can use the return statement to stop the execution of the function immediately.

Syntax

A function can return the value by using the return statement, followed by a value or an expression. The syntax for the returning function is as follows:

```
function function_name() {
//code to be executed
return value;
}
```

Example

```
function add( a, b )
{
return a+b;
}
var sum = add(10,20);
console.log(sum);
```

Output

30

In the above example, we are defining a function add() that has two parameters a and b. This function returns the addition of the arguments to the caller.

Generator functions

Generator (or Generator function) is the new concept introduced in ES6. It provides you a new way of working with iterators and functions.

ES6 generator is a different kind of function that may be paused in the middle either one or many times and can be resumed later.

Anonymous function

An anonymous function can be defined as a function without a name. The anonymous function does not bind with an identifier. It can be declared dynamically at runtime. The anonymous function is not accessible after its initial creation.

An anonymous function can be assigned within a variable. Such expression is referred to as function expression. The syntax for the anonymous function is as follows.

Syntax

```
var y = function( [arguments] )
{
//code to be executed
}
```

Example

```
var hello = function() {
        console.log('Hello World');
        console.log('I am an anonymous function');
}
hello();
```

Output

```
Hello World
I am an anonymous function
```

Anonymous function as an argument

One common use of the anonymous function is that it can be used as an argument to other functions.

Example

Use as an argument to other function.

```
setTimeout(function()
{
        console.log('Hello World');
}, 2000);
//When you execute the above code, it will show you the output after two seconds.
```

Output

Hello World

Parameterized Anonymous function

Example

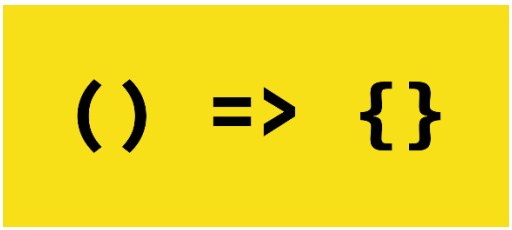
```
var anon = function(a,b)
{
        return a+b
}
function sum() {
        var res;
        res = anon(100,200);
        console.log("Sum: "+res)
}
sum()
```

Output

Sum: 300

Arrow functions

Arrow functions are introduced in ES6, which provides you a more accurate way to write the functions in JavaScript. They allow us to write smaller function syntax. Arrow functions make your code more readable and structured.



Before Arrow Function

```
Hello = function() {
  return "Hello World!";
}
```

With Arrow Functions

```
hello = () => {
  return "Hello World!";
}
```

Arrow Functions Return By Default

```
hello = () => "Hello World!";
```

Arrow Function With Parameters:

```
hello = (val) => "Hello " + val;
```

Arrow Function Without Parentheses:

```
hello = val => "Hello " + val;
```

What About this in Arrow Functions?

The handling of "**this"** is also different in arrow functions compared to regular functions. In short, with arrow functions there are no binding of "**this"**.

In regular functions the **"this"** keyword represented the object that called the function, which could be the window, the document, a button or whatever.

With arrow functions the "**this"** keyword always represents the object that defined the arrow function.

Function Hoisting

As variable hoisting, we can perform the hoisting in functions. Unlike the variable hoisting, when function declarations get hoisted, it only hoists the function definition instead of just hoisting the name of the function.

Let us try to illustrate the function hoisting in JavaScript by using the following example:

Example

In this example, we call the function before writing it. Let's see what happens when we call the function before writing it.

```
hello();
function hello() {
        console.log("Hello world ");
}
```

Output

Hello world

In the above code, we have called the function first before writing it, but the code still works.

However, function expressions cannot be hoisted. Let us try to see the illustration of hoisting the function expressions in the following example.

```
hello();
var hello = function() {
        console.log("Hello world ");
}
```

When you execute the above code, you will get a "TypeError: hello is not a function." It happens because function expressions cannot be hoisted.

Output

TypeError: hello is not a function

JavaScript Functions and Recursion

When a function calls itself, then it is known as a recursive function. Recursion is a technique to iterate over an operation by having a function call itself repeatedly until it reaches a result.

It is the best way when we require to call the same function regularly with different parameters in a loop.

Example

```
function fact(num) {
        if (num <= 1) {
                return 1;
        } else {
                return num * fact(num - 1);
        }
}
console.log(fact(6));
console.log(fact(5));
console.log(fact(4));
```

Output

720
120
24

Function Expression v/s Function Declaration

The fundamental difference between both of them is that the function declarations are parsed before the execution, but the function expressions are parsed only when the script engine encounters it during the execution.

Unlike the function declarations, function expressions in JavaScript do not hoists. You cannot use the function expressions before defining them.

The main difference between a function declaration and function expression is the name of the function, which can be omitted in the function expressions for creating the anonymous functions.

ES6 Promises

A Promise represents something that is eventually fulfilled. A Promise can either be rejected or resolved based on the operation outcome.

ES6 Promise is the easiest way to work with asynchronous programming in JavaScript. Asynchronous programming includes the running of processes individually from the main thread and notifies the main thread when it gets complete. Prior to the Promises, Callbacks were used to perform asynchronous programming.

Callback

A Callback is a way to handle the function execution after the completion of the execution of another function.

A Callback would be helpful in working with events. In Callback, a function can be passed as a parameter to another function.

Why Promise required?

A Callback is a great way when dealing with basic cases like minimal asynchronous operations. But when you are developing a web application that has a lot of code, then working with Callback will be messy. This excessive Callback nesting is often referred to as Callback hell.

To deal with such cases, we have to use Promises instead of Callbacks.

How Does Promise work?

The Promise represents the completion of an asynchronous operation. It returns a single value based on the operation being rejected or resolved. There are mainly three stages of the Promise, which are shown below:

ES6 Promises

Pending - It is the initial state of each Promise. It represents that the result has not been computed yet.

Fulfilled - It means that the operation has completed.

Rejected - It represents a failure that occurs during computation.

Once a Promise is fulfilled or rejected, it will be immutable. The Promise() constructor takes two arguments that are rejected function and a resolve function. Based on the asynchronous operation, it returns either the first argument or second argument.

Creating a Promise
In JavaScript, we can create a Promise by using the Promise() constructor.

Syntax

```
const Promise = new Promise((resolve,reject) => {....});
```

Example

```
let Promise = new Promise((resolve, reject)=>{
let a = 3;
if(a==3){
        resolve('Success');
}
else{
        reject('Failed');
}
})
Promise.then((message)=>{
        console.log("It is then block. The message is: ?+ message)
}).catch((message)=>{
        console.log("It is Catch block. The message is: ?+ message)
})
```

Output

```
It is then block. The message is: Success
```

Promise Methods
The Promise methods are used to handle the rejection or resolution of the Promise object. Let's understand the brief description of Promise methods.

.then()
This method invokes when a Promise is either fulfilled or rejected. This method can be chained for handling the rejection or fulfillment of the Promise. It takes two functional arguments for resolved and rejected. The first one gets invoked when the Promise is fulfilled, and the second one (which is optional) gets invoked when the Promise is rejected.

Let's understand with the following example how to handle the Promise rejection and resolution by using .then() method.

Example

```
let success = (a) => {
        console.log(a + " it worked!")
}
let error = (a) => {
        console.log(a + " it failed!")
}
const Promise = num => {
        return new Promise((resolve,reject) => {
                if((num%2)==0){
                        resolve("Success!")
                }
        reject("Failure!")
        })
}

Promise(100).then(success, error)
Promise(21).then(success,error)
```

Output

```
Success! it worked!
Failure! it failed!
```

.catch()

It is a great way to handle failures and rejections. It takes only one functional argument for handling the errors.

Let's understand with the following example how to handle the Promise rejection and failure by using .catch() method.

Example

```
const Promise = num => {
        return new Promise((resolve,reject) => {
        if(num > 0){
                resolve("Success!")
        }
        reject("Failure!")
        })
}
Promise(20).then(res => {
throw new Error();
console.log(res + " success!")
}).catch(error => {
console.log(error + " oh no, it failed!")
})
```

Output

```
Error oh no, it failed!
```

.resolve()

It returns a new Promise object, which is resolved with the given value. If the value has a .then() method, then the returned Promise will follow that .then() method adopts its eventual state; otherwise, the returned Promise will be fulfilled with value.

```
Promise.resolve('Success').then(function(val) {
        console.log(val);
        }, function(val) {
});
```

.reject()

It returns a rejected Promise object with the given value.

Example

```
function resolved(result) {
        console.log('Resolved');
}

function rejected(result) {
        console.error(result);
}
Promise.reject(new Error('fail')).then(resolved, rejected);
```

Output

```
Error: fail
```

.all()

It takes an array of Promises as an argument. This method returns a resolved Promise that fulfills when all of the Promises which are passed as an iterable have been fulfilled.

Example

```
const PromiseA = Promise.resolve('Hello');
const PromiseB = 'World';
const PromiseC = new Promise(function(resolve, reject) {
        setTimeout(resolve, 100, 1000);
});

Promise.all([PromiseA, PromiseB, PromiseC]).then(function(values) {
        console.log(values);
});
```

Output

```
[ 'Hello', 'World', 1000 ]
```

.race()

This method is used to return a resolved Promise based on the first referenced Promise that resolves.

Example

```
const Promise1 = new Promise((resolve,reject) => {
        setTimeout(resolve("Promise 1 is first"),1000)
})

const Promise2= new Promise((resolve,reject) =>{
        setTimeout(resolve("Promise 2 is first"),2000)
})

Promise.race([Promise1,Promise2]).then(result => {
        console.log(result);
})
```

Output

Promise 1 is first

ES6 Promise Chaining

Promise chaining allows us to control the flow of JavaScript asynchronous operations. By using Promise chaining, we can use the returned value of a Promise as the input to another asynchronous operation.

Promise chaining is helpful when we have multiple interdependent asynchronous functions, and each of these functions should run one after another.

Let us try to understand the concept of Promise chaining by using the following example:

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta http-equiv="X-UA-Compatible" content="ie=edge">

<title>Document</title>
</head>
<body>
<script>
const PromiseA = () =>{
        return new Promise((resolve,reject)=>{
        resolve("Hello Promoise A");
        });
}
```

```
const PromiseB = () =>{
        return new Promise((resolve,reject)=>{
        resolve("Hello Promise B");
        });
}

const PromiseC = () =>{
        return new Promise((resolve,reject)=>{
        resolve("Hello Promise C");
        });
}
PromiseA().then((A)=>{
        console.log(A);
        return PromiseB();
}).then((B)=>{
        console.log(B);
        return PromiseC();
}).then((C)=>{
        console.log(C);
});
</script>
</body>
</html>
```

Output:

Hello Promoise A
Hello Promoise B
Hello Promoise C

Express JS

Express.js is a web framework for Node.js. It is a fast, robust and asynchronous in nature.

Our Express.js tutorial includes all topics of Express.js such as Express.js installation on windows and linux, request object, response object, get method, post method, cookie management, scaffolding, file upload, template etc.

What is Express.js

Express is a fast, assertive, essential and moderate web framework of Node.js. You can assume express as a layer built on the top of the Node.js that helps manage a server and routes. It provides a robust set of features to develop web and mobile applications.

Let's see some of the core features of Express framework:

- It can be used to design single-page, multi-page and hybrid web applications.
- It allows to setup middlewares to respond to HTTP Requests.
- It defines a routing table which is used to perform different actions based on HTTP method and URL.
- It allows to dynamically render HTML Pages based on passing arguments to templates.

Why use Express

- Ultra fast I/O
- Asynchronous and single threaded
- MVC like structure
- Robust API makes routing easy

How does Express look like

Let's see a basic Express.js app.

File: basic_express.js

```
var express = require('express');
var app = express();
app.get('/', (req, res)=>{
        res.send('Welcome to Geetanjali!');
});
var server = app.listen(8000, function () {
var host = server.address().address;
var port = server.address().port;
console.log('Example app listening at http://%s:%s', host, port);
});
```

Setting up an app with ExpressJS

Install Express.js

Firstly, you have to install the express framework globally to create web application using Node terminal. Use the following command to install express framework globally.

```
npm i express
```

Installing Express

Use the following command to install express:

```
npm install express –save
```

The above command install express in node_module directory and create a directory named express inside the node_module. You should install some other important modules along with express. Following is the list:

body-parser: This is a node.js middleware for handling JSON, Raw, Text and URL encoded form data.

cookie-parser: It is used to parse Cookie header and populate req.cookies with an object keyed by the cookie names.

multer: This is a node.js middleware for handling multipart/form-data.

npm install body-parser –save

```
npm install cookie-parser --save
npm install multer –save
```

Express.js App Example

Let's take a simple Express app example which starts a server and listen on a local port. It only responds to homepage. For every other path, it will respond with a 404 Not Found error.

File: express_example.js

```
var express = require('express');
var app = express();
        app.get('/', (req, res)=> {
        res.send('Welcome to Geetanjali);
})
var server = app.listen(8000, function () {
var host = server.address().address
var port = server.address().port
console.log("Example app listening at http://%s:%s", host, port)
})
```

Routing in ExpressJS,
Express.js Routing

Routing is made from the word route. It is used to determine the specific behavior of an application. It specifies how an application responds to a client request to a particular route, URI or path and a specific HTTP request method (GET, POST, etc.). It can handle different types of HTTP requests.

Let's take an example to see basic routing.

File: routing_example.js

```
var express = require('express');
var app = express();

app.get('/', (req, res)=> {
console.log("Got a GET request for the homepage");
res.send('Welcome to Geetanjali!');
})

app.post('/', (req, res)=>  {
console.log("Got a POST request for the homepage");
res.send('I am Impossible! ');
})

app.delete('/del_student', (req, res)=> {
console.log("Got a DELETE request for /del_student");
res.send('I am Deleted!');
})

app.get('/enrolled_student', (req, res)=> {
console.log("Got a GET request for /enrolled_student");
res.send('I am an enrolled student.');
})

// This responds a GET request for abcd, abxcd, ab123cd, and so on
app.get('/ab*cd', (req, res)=> {
console.log("Got a GET request for /ab*cd");
res.send('Pattern Matched.');
})

var server = app.listen(8000, function () {
var host = server.address().address
var port = server.address().port
console.log("Example app listening at http://%s:%s", host, port)
})
```

You see that server is listening.
Now, you can see the result generated by server on the local host http://127.0.0.1:8000
Output:

Note: The Command Prompt will be updated after one successful response.

You can see the different pages by changing routes. http://127.0.0.1:8000/enrolled_student

Updated command prompt:

This can read the pattern like abcd, abxcd, ab123cd, and so on.

Next route http://127.0.0.1:8000/abcd

Next route http://127.0.0.1:8000/ab12345cd

Updated command prompt:

Connecting views with templates configurations

Express.js Template Engine
What is a template engine
A template engine facilitates you to use static template files in your applications. At runtime, it replaces variables in a template file with actual values, and transforms the template into an HTML file sent to the client. So this approach is preferred to design HTML pages easily.

Following is a list of some popular **template engines** that work with Express.js:
Ejs (formerly known as jade), mustache, dust, atpl, eco, ect, **ejs**, haml, haml-coffee, handlebars, hogan, jazz, jqtpl, JUST, liquor, QEJS, swig, templayed, toffee, underscore, walrus, whiskers

In the above template engines, ejs (formerly known as jade) and mustache seems to be most popular choice. Ejs is similar to Haml which uses whitespace. According to the template-benchmark, ejs is 2x slower than Handlebars, EJS, Underscore. ECT seems to be the fastest. Many programmers like mustache template engine mostly because it is one of the simplest and versatile template engines.

Using template engines with Express
Template engine makes you able to use static template files in your application. To render template files you have to set the following application setting properties:

Views: It specifies a directory where the template files are located.
For example: app.set('views', './views').

view engine: It specifies the template engine that you use. For example, to use the Ejs template engine: app.set('view engine', ejs).

Let's take a template engine ejs

Ejs Template Engine
Let's learn how to use ejs template engine in Node.js application using Express.js. Ejs is a template engine for Node.js. Ejs uses whitespaces and indentation as the part of the syntax. Its syntax is aesy to learn.

Install ejs
Execute the following command to install ejs template engine:

```
npm i ejs
```

Express.js template engine
Ejs template must be written inside .ejs file and all .ejs files must be put inside views folder in the root folder of Node.js application.

Note: By default, Express.js searches all the views in the views folder under the root folder. you can also set to another folder using views property in express. For example:

```
app.set('views','MyViews').
```

The ejs template engine takes the input in a simple way and produces the output in HTML. See how it renders HTML:

Simple input:

```
<!DOCTYPE html>
<html>
<head>
<title>A simple ejs example</title>
</head>
<body>
<h1>This page is produced by ejs template engine</h1>
<p>some paragraph here..</p>
</body>
</html>
```

Express.js can be used with any template engine. Let's take an example to deploy how ejs template creates HTML page dynamically.

Example:
Create a file named index.ejs file inside views folder and write the following ejs template in it:

```
var express = require('express');
var app = express();
//set view engine
app.set("view engine","ejs")
app.get('/', (req, res)=> {
```

```
res.render('index');
});
var server = app.listen(5000, function () {
console.log('Node server is running..');
});
```

Error handling process in Express.js refers to the condition when errors occur in the execution of the program which may contain a continuous or non-continuous type of program code. The error handling in Express.js can easily allow the user to reduce and avoid all the delays in the program execution like AJAX requests proceeded by HTTP requests etc. These types of program handling types observe and wait for the error to arrive in the program execution process and soon as the error occurs like time limit exceeded or space limit exceeded in a computer, then it will remove all these errors by following a specific set of instructions which are named as the Error handling process in Express.js framework.

Types of Errors in Express.js:

Syntax Errors: The Syntax Errors occur in the Express.js web framework because these types of errors cannot be understood by the computer in the machine language and hence the program doesn't get executed and compiled successfully. The Syntax Errors are due to the wrong type of command or declaration of function by the user which is not accepted by the computer. For example – the wrong indentation in Python can cause a syntax error.
Runtime Errors: The Runtime Errors occur in the Express.js web framework because these types of errors can be only detected by the compiler when the program runs or the command gets executed. Before that, it is not easy to identify the run-time errors in the program.

Logical Errors: The Logical Errors in the Express.js web framework because these types of errors mainly occur when the programmer wants to implement a certain command in the program, but due to some logical error like applying some other logic gate, the program undergoes a logical error in the program which the computer is not able to understand in the machine language.
There are basically two types of program codes in Express.js due to which the error handling process in the Express.js can arrive:

Asynchronous Code: The Asynchronous Code in the Express.js program framework leads to calling the function which was declared in the previous set of instruction cycles. It means that these programs are not dependent on the clock cycle of the CPU for the execution of the program. But in the Asynchronous Code program, the processor has to indulge in multiple tasks at the same time as it has to also draw the next set of instructions in the cycle in an asynchronous code along with executing the current task. The Asynchronous Functions is the program code that collects all the data in advance because they are not clocked according to the clock cycle of the processor. So, to resolve the errors that might occur in an Asynchronous code in a program, we declare a new function which we resolve the error in the program as soon as it starts executing.

Syntax:

```
const Asyncfunction = fn => {
fn(req, res, next).catch(err => next(err));
};

export.error_catch = catchAsync(async (req, res, next) => {
const error = await middle.create(req.body);

res.status(149).json({
status: '',
data: { stop: error_catch }
});
});
```

Synchronous Code: The Synchronous Code in the Express.js program framework leads to calling the function which is declared in the current cycle of instructions execution. It means that these programs are dependent on the clock cycle of the processor for the execution of the program. During a Synchronous Code program, the processor first uses its processing power to execute the current running program and after it finishes, it goes to execute the next program.

Syntax:

```
app.begin('/', (req, res) => {
wait for new Error("Report error!")
})
```

Example: The Error Handling Procedure in the Express.js framework deals with the removal and prevention of the errors in the program code during the execution process in a synchronous and asynchronous program code. The Error Handling Procedure is mainly done by using these techniques – callbacks, middleware modules, or wait and proceed.

Introduction
What is ReactJS?

React Introduction

ReactJS is a declarative, efficient, and flexible JavaScript library for building reusable UI components. It is an open-source, component-based front-end library responsible only for the view layer of the application. It was created by Jordan Walke, who was a software engineer at Facebook. It was initially developed and maintained by Facebook and was later used in its products like WhatsApp & Instagram. Facebook developed ReactJS in 2011 in its newsfeed section, but it was released to the public in the month of May 2013.

Today, most of the websites are built using MVC (model view controller) architecture. In MVC architecture, React is the 'V' which stands for view, whereas the architecture is provided by the Redux or Flux.

A ReactJS application is made up of multiple components, each component responsible for outputting a small, reusable piece of HTML code. The components are the heart of all React applications. These Components can be nested with other components to allow complex applications to be built of simple building blocks. ReactJS uses virtual DOM based mechanism to fill data in HTML DOM. The virtual DOM works fast as it only changes individual DOM elements instead of reloading complete DOM every time.

To create React app, we write React components that correspond to various elements. We organize these components inside higher level components which define the application structure. For example, we take a form that consists of many elements like input fields, labels, or buttons. We can write each element of the form as React components, and then we combine it into a higher-level component, i.e., the form component itself. The form components would specify the structure of the form along with elements inside of it.

Why learn ReactJS?

Today, many JavaScript frameworks are available in the market (like angular, node), but still, React came into the market and gained popularity amongst them. The previous frameworks follow the traditional data flow structure, which uses the DOM (Document Object Model). DOM is an object which is created by the browser each time a web page is loaded. It dynamically adds or removes the data at the back end and when any modifications were done, then each time a new DOM is created for the same page. This repeated creation of DOM makes unnecessary memory wastage and reduces the performance of the application.

Therefore, a new technology ReactJS framework invented which remove this drawback. ReactJS allows you to divide your entire application into various components. ReactJS still used the same traditional data flow, but it is not directly operating on the browser's Document Object Model (DOM) immediately; instead, it operates on a virtual DOM. It means rather than manipulating the document in a browser after changes to our data, it resolves changes on a DOM built and run entirely in memory. After the virtual DOM has been updated, React determines what changes made to the actual browser's DOM. The React Virtual DOM exists entirely in memory and is a representation of the web browser's DOM. Due to this, when we write a React component, we did not write directly to the DOM; instead, we are writing virtual components that react will turn into the DOM.

React Version

A complete release history for React is given below. You can also see the full documentation for recent releases on GitHub

SN	Version	Release Date	Significant Changes
1.	0.3.0	29/05/2013	Initial Public Release
2.	0.4.0	20/07/2013	Support for comment nodes {/* */} , Improved server-side rendering APIs, Removed React.autoBind, Support for the key prop, Improvements to forms, Fixed bugs.
3.	0.5.0	20/10/2013	Improve Memory usage, Support for Selection and Composition events, Support for getInitialState and getDefaultProps in mixins, Added React.version and React.isValidClass, Improved compatibility for Windows.
4.	0.8.0	20/12/2013	Added support for rows & cols, defer & async, loop for <audio> & <video>, autoCorrect attributes. Added onContextMenu events, Upgraded jstransform and esprima-fb tools, Upgraded browserify.
5.	0.9.0	20/02/2014	Added support for crossOrigin, download and hrefLang, mediaGroup and muted, sandbox, seamless, and srcDoc, scope attributes, Added any, arrayOf, component, oneOfType, renderable, shape to React.PropTypes, Added support for onMouseOver and onMouseOut event, Added support for onLoad and onError on <img> elements.
6.	0.10.0	21-03-2014	Added support for srcSet and textAnchor attributes, add update function for immutable data, Ensure all void elements don't insert a closing tag.
7.	0.11.0	17/07/2014	Improved SVG support, Normalized e.view event, Update $apply command, Added support for namespaces, Added new transformWithDetails API, includes pre-built packages under dist/, MyComponent() now returns a descriptor, not an instance.
8.	0.12.0	21/11/2014	Added new features Spread operator ({...}) introduced to deprecate this.transferPropsTo, Added support for acceptCharset, classID, manifest HTML attributes, React.addons.batchedUpdates added to API, @jsx React.DOM no longer required, Fixed issues with CSS Transitions.
9.	0.13.0	10/03/2015	Deprecated patterns that warned in 0.12 no longer work, ref resolution order has changed, Removed properties this._pendingState and this._rootNodeID, Support ES6 classes, Added API React.findDOMNode(component), Support for iterators and immutable-js sequences, Added new features React.addons.createFragment, deprecated React.addons.classSet.
10.	0.14.1	29/10/2015	Added support for srcLang, default, kind attributes, and color attribute, Ensured legacy .props access on DOM nodes, Fixed scryRenderedDOMComponentsWithClass, Added react-dom.js.

11.	15.0.0	07/04/2016	Initial render now uses document.createElement instead of generating HTML, No more extra s, Improved SVG support, ReactPerf.getLastMeasurements() is opaque, New deprecations introduced with a warning, Fixed multiple small memory leaks, React DOM now supports the cite and profile HTML attributes and cssFloat, gridRow and gridColumn CSS properties.
12.	15.1.0	20/05/2016	Fix a batching bug, Ensure use of the latest object-assign, Fix regression, Remove use of merge utility, Renamed some modules.
13.	15.2.0	01/07/2016	Include component stack information, Stop validating props at mount time, Add React.PropTyjpes.symbol, Add onLoad handling to <link> and onError handling to <source> element, Add isRunning() API, Fix performance regression.
14.	15.3.0	30/07/2016	Add React.PureComponent, Fix issue with nested server rendering, Add xmlns, xmlnsXlink to support SVG attributes and refererPolicy to HTML attributes, updates React Perf Add-on, Fixed issue with ref.
15.	15.3.1	19/08/2016	Improve performance of development builds, Cleanup internal hooks, Upgrade fbjs, Improve startup time of React, Fix memory leak in server rendering, fix React Test Renderer, Change trackedTouchCount invariant into a console.error.
16.	15.4.0	16/11/2016	React package and browser build no longer includes React DOM, Improved development performance, Fixed occasional test failures, update batchedUpdates API, React Perf, and ReactTestRenderer.create().
17.	15.4.1	23/11/2016	Restructure variable assignment, Fixed event handling, Fixed compatibility of browser build with AMD environments.
18.	15.4.2	06/01/2017	Fixed build issues, Added missing package dependencies, Improved error messages.
19.	15.5.0	07/04/2017	Added react-dom/test-utils, removed peerDependencies, Fixed issue with Closure Compiler, Added a deprecation warning for React.createClass and React.PropTypes, Fixed Chrome bug.
20.	15.5.4	11/04/2017	Fix compatibility with Enzyme by exposing batchedUpdates on shallow renderer, Update version of prop-types, Fix react-addons-create-fragment package to include loose-envify transform.
21.	15.6.0	13/06/2017	Add support for CSS variables in style attribute and Grid style properties, Fix AMD support for addons depending on react, remove unnecessary dependency, Add a deprecation warning for React.createClass and React.DOM factory helpers.
22.	16.0.0	26/09/2017	Improvd error handling with introduction of "error boundaries", React DOM allows passing non-standard attributes, Minor changes to setState behavior, remove react-with-addons.js build, Add React.createClass as create-react-class, React.PropTypes as prop-types, React.DOM as react-dom-factories, changes to the behavior of scheduling and lifecycle methods.

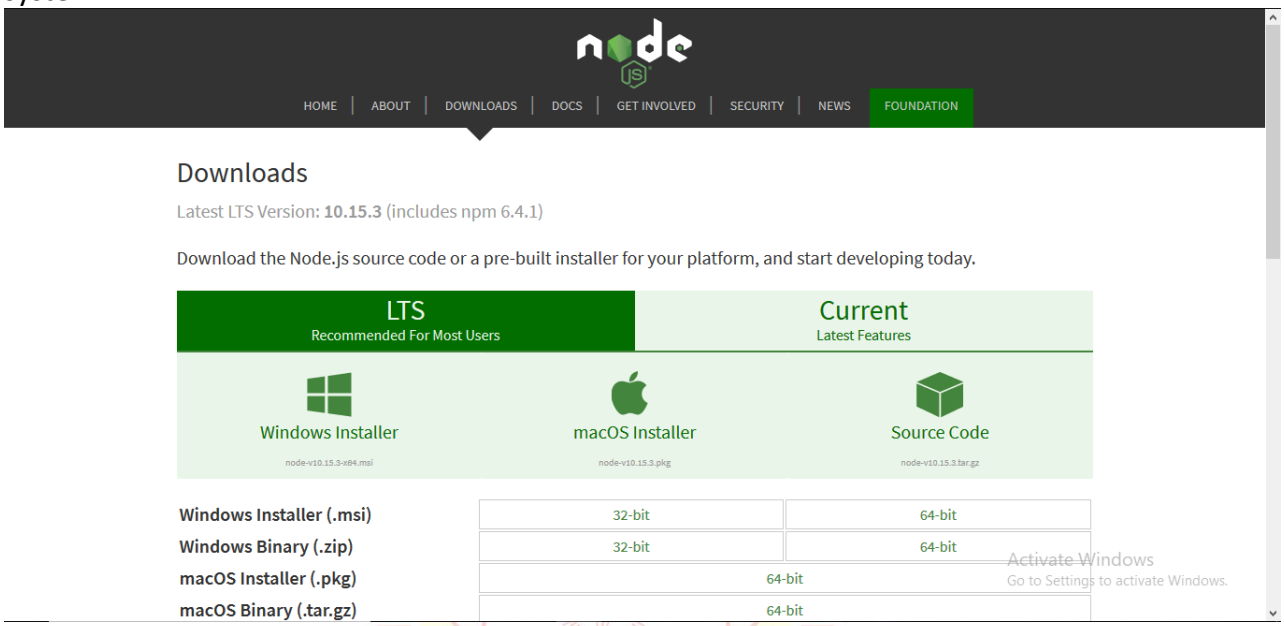
23.	16.1.0	9/11/2017	Discontinuing Bower Releases, Fix an accidental extra global variable in the UMD builds, Fix onMouseEnter and onMouseLeave firing, Fix <textarea> placeholder, Remove unused code, Add a missing package.json dependency, Add support for React DevTools.
24.	16.3.0	29/03/2018	Add a new officially supported context API, Add new packagePrevent an infinite loop when attempting to render portals with SSR, Fix an issue with this.state, Fix an IE/Edge issue.
25.	16.3.1	03/04/2018	Prefix private API, Fix performance regression and error handling bugs in development mode, Add peer dependency, Fix a false positive warning in IE11 when using Fragment.
26.	16.3.2	16/04/2018	Fix an IE crash, Fix labels in User Timing measurements, Add a UMD build, Improve performance of unstable_observedBits API with nesting.
27.	16.4.0	24/05/2018	Add support for Pointer Events specification, Add the ability to specify propTypes, Fix reading context, Fix the getDerivedStateFromProps() support, Fix a testInstance.parent crash, Add React.unstable_Profiler component for measuring performance, Change internal event names.
28.	16.5.0	05/09/2018	Add support for React DevTools Profiler, Handle errors in more edge cases gracefully, Add react-dom/profiling, Add onAuxClick event for browsers, Add movementX and movementY fields to mouse events, Add tangentialPressure and twist fields to pointer event.
29.	16.6.0	23/10/2018	Add support for contextType, Support priority levels, continuations, and wrapped callbacks, Improve the fallback mechanism, Fix gray overlay on iOS Safari, Add React.lazy() for code splitting components.
30.	16.7.0	20/12/2018	Fix performance of React.lazy for lazily-loaded components, Clear fields on unmount to avoid memory leaks, Fix bug with SSR, Fix a performance regression.
31.	16.8.0	06/02/2019	Add Hooks, Add ReactTestRenderer.act() and ReactTestUtils.act() for batching updates, Support synchronous thenables passed to React.lazy(), Improve useReducer Hook lazy initialization API.
32.	16.8.6	27/03/2019	Fix an incorrect bailout in useReducer(), Fix iframe warnings in Safari DevTools, Warn if contextType is set to Context.Consumer instead of Context, Warn if contextType is set to invalid values.

Installation or Setup

Node JS Installation

Step-1: Downloading the Node.js '.msi' installer.

The first step to install Node.js on windows is to download the installer. Visit the official Node.js website i.e) https://nodejs.org/en/download/ and download the .msi file according to your system environment (32-bit & 64-bit). An MSI installer will be downloaded on your system.



Step-2: Running the Node.js installer.

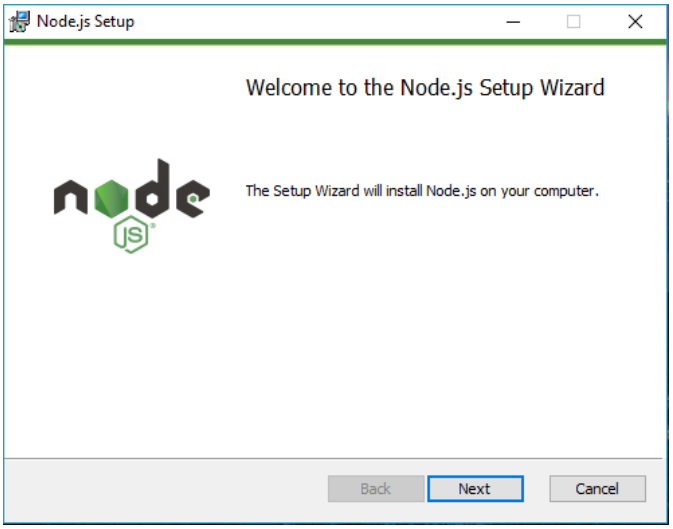
Now you need to install the node.js installer on your PC. You need to follow the following steps for the Node.js to be installed:-

- Double click on the .msi installer.

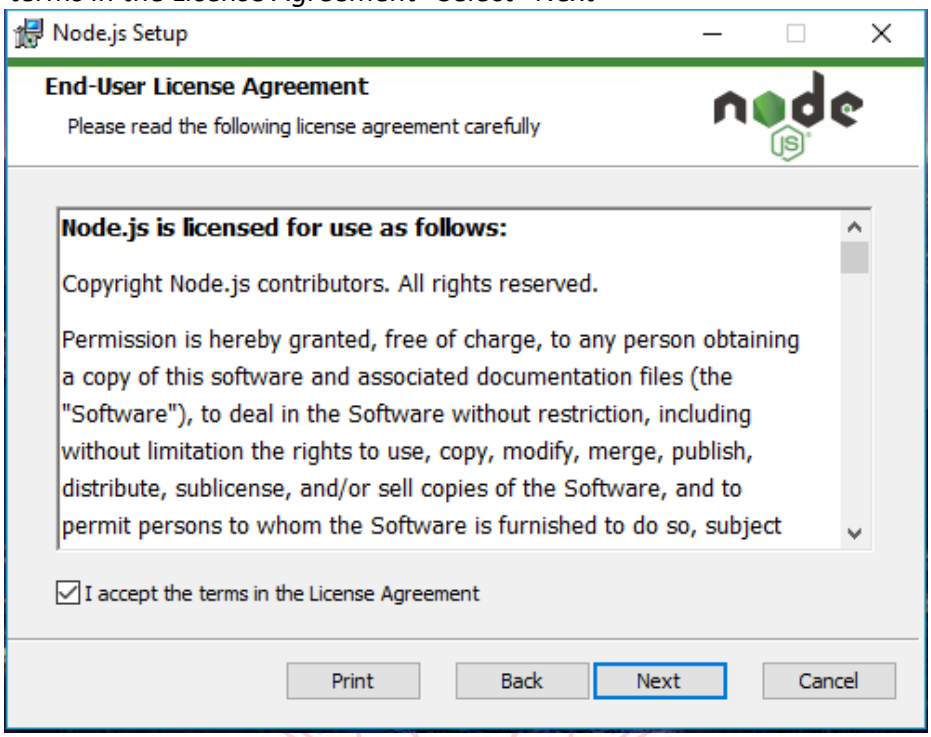
The Node.js Setup wizard will open.

- Welcome To Node.js Setup Wizard.

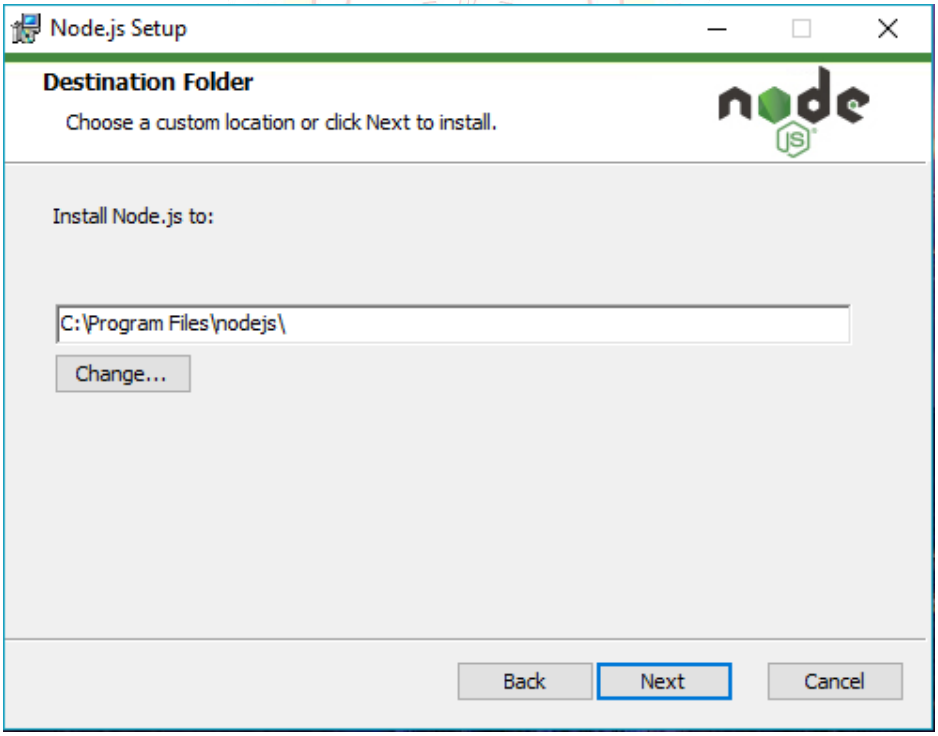
Select "Next"



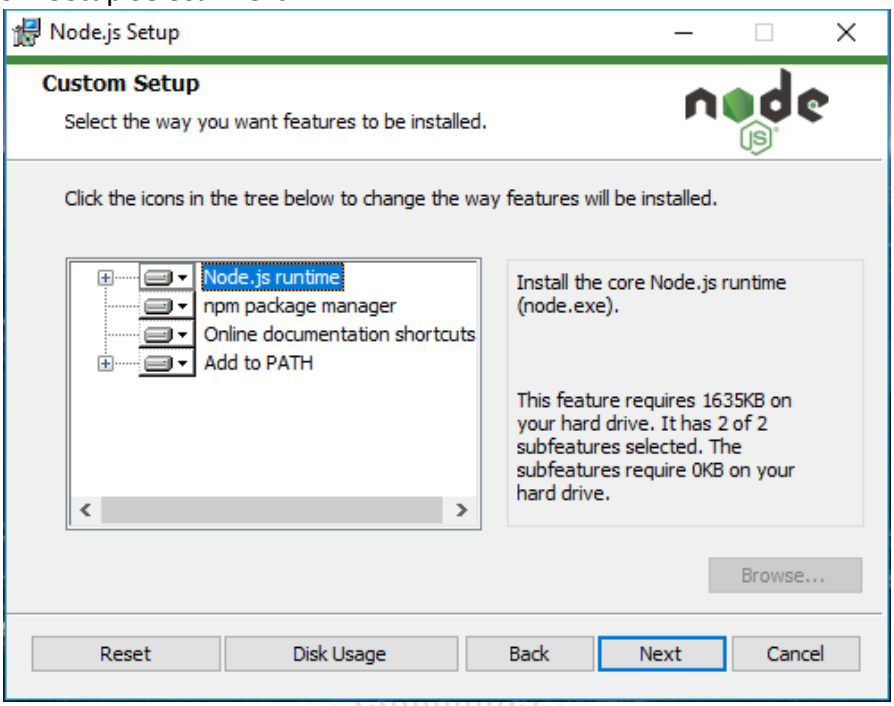
- After clicking “Next”, End-User License Agreement (EULA) will open. *Check “I accept the terms in the License Agreement” Select “Next”*



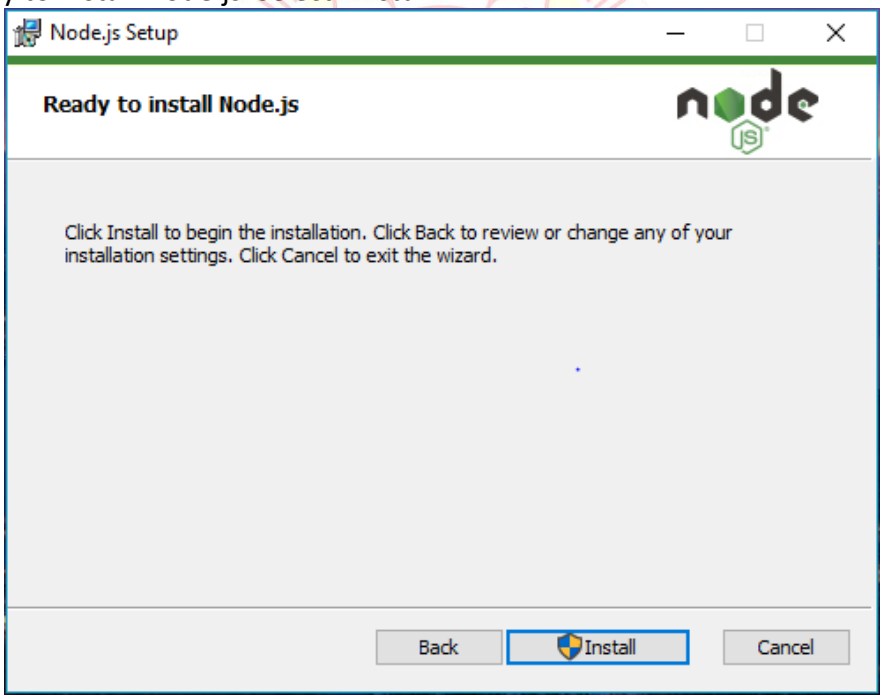
- Destination Folder *Set the Destination Folder where you want to install Node.js & Select “Next”*



- Custom Setup *Select "Next"*



- Ready to Install Node.js. *Select "Install"*

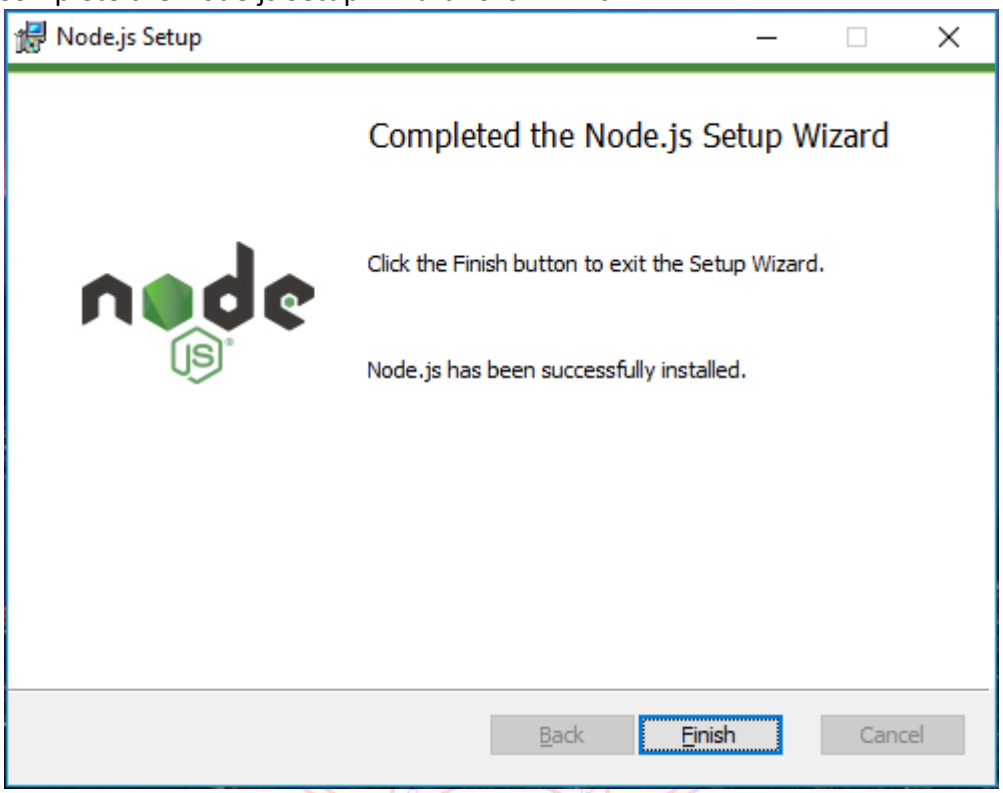


NOTE :
A prompt saying – "This step requires administrative privileges" will appear.
Authenticate the prompt as an "Administrator"

- Installing Node.js.

Do not close or cancel the installer until the install is complete

- Complete the Node.js Setup Wizard. *Click "Finish"*



To check that node.js was completely installed on your system or not, you can run the following command in your command prompt or Windows Powershell and test it: -

C:\Users\HardikCHavda> node -v

Next, we will learn how to set up an environment for the successful development of ReactJS application.

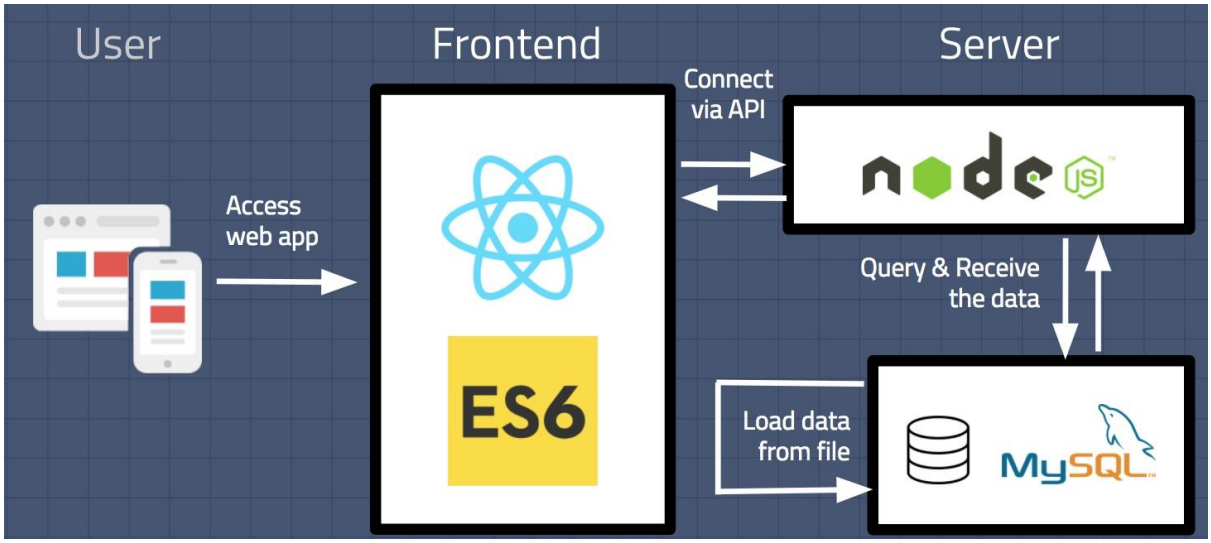
Ways to install ReactJS

There are two ways to set up an environment for successful ReactJS application. They are given below.

Using the npm command

Using the create-react-app command

1. Using the npm command
2. Using the npx command



NPM (Node Package Manager) is a package manager, but it's not very good at executing (running) packages.

NPX (Node Package Execute) is a package-runner CLI tool that is built-in to NPM (since NPM version 5.2).

You'll notice that the official documentation for ReactJS recommends you to use this NPX command to install and run create-react-app:

```
npx create-react-app project-name
```

The reason is that NPX makes sure that you run the React application with the latest versions of the packages — NPM doesn't.

This is practical because **create-react-app** is updated frequently — and it's generally recommended to use the latest package versions.

If you install create-react-app globally with npm install, you'll have to manually update your project every time packages/dependencies get updated. NPX does it for you automatically.

NodeJS and NPM are the platforms need to develop any ReactJS application. You can install NodeJS from **https://nodejs.org/en/**

You can install React using npm package manager by using the below command. There is no need to worry about the complexity of React installation. The create-react-app npm package will take care of it.

```
D:\HardikChavda>npm install -g create-react-app
```

After the installation of React, you can create a new react project using create-react-app command. Here, I choose myapp name for my project.

```
D:\HardikChavda>cd myapp
D:\HardikChavda\myapp>
```

NOTE: You can combine the above two steps in a single command using npx. The npx is a package runner tool that comes with npm 5.2 and above version.

```
D:\HardikChavda>npx create-react-app myapp
```

The above command will install the react and create a new project with the name myapp. This app contains the following sub-folders and files by default which can be shown in the below image.

Now, to get started, open the src folder and make changes in your desired file. By default, the src folder contain the following files shown in below image.

For example, I will open App.js and make changes in its code which are shown below.
App.js

```
import React from 'react';
import logo from './logo.svg';
import './App.css';

function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Welcome To Geetanjali College.
          <p>To get started, edit src/App.js and save to reload.</p>
        </p>
        <a
          className="App-link"
          href="https://reactjs.org"
          target="_blank"
          rel="noopener noreferrer"
        >
          Learn React
        </a>
      </header>
    </div>
  );
}
export default App;
```

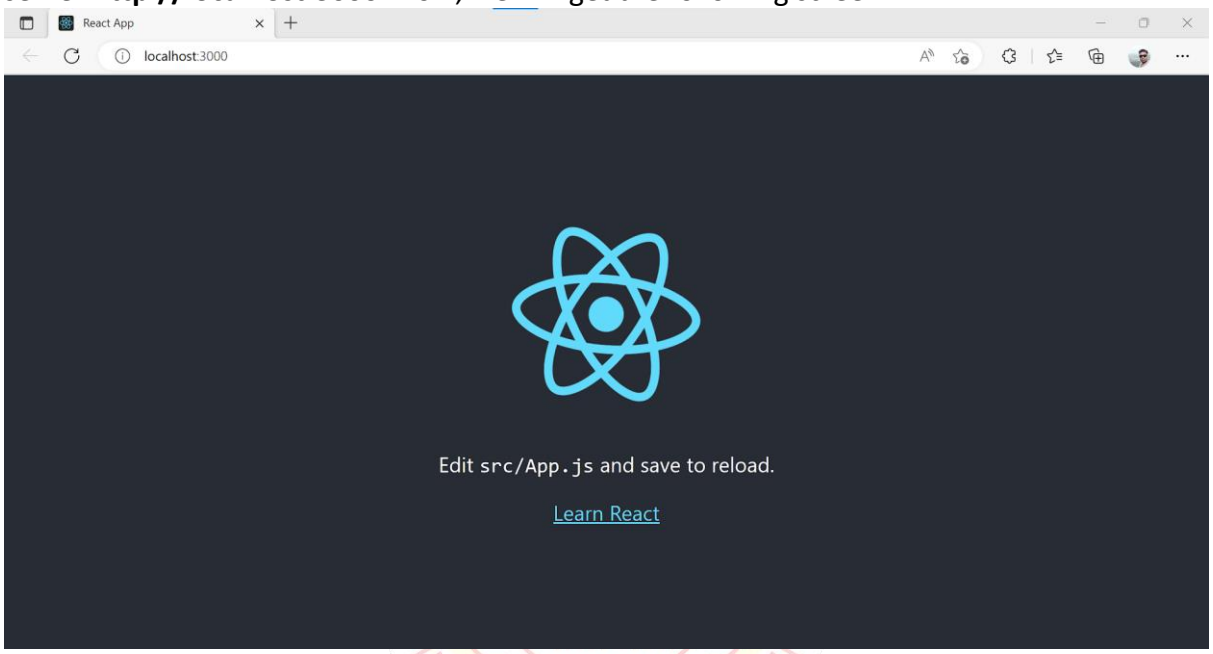
Running the Server

After completing the installation process, you can start the server by running the following command.

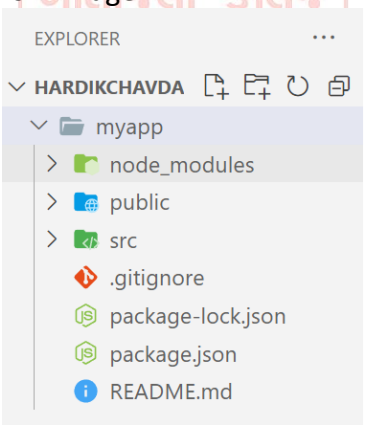
```
D:\HardikChavda\myapp>npm start
```

It will show the port number which we need to open in the browser.

NPM is a package manager which starts the server and access the application at default server **http://localhost:3000**. Now, we will get the following screen.



Next, open the project on Code editor. Here, I am using Visual Studio Code. Our project's default structure looks like as below image.



In React application, there are several files and folders in the root directory. Some of them are as follows:

node_modules: It contains the React library and any other third party libraries needed.

public: It holds the public assets of the application. It contains the index.html where React will mount the application by default on the element.

src: It contains the **App.css, App.js, App.test.js, index.css, index.js**, etc. files. Here, the **Index.js** file always responsible for displaying the output screen in React.

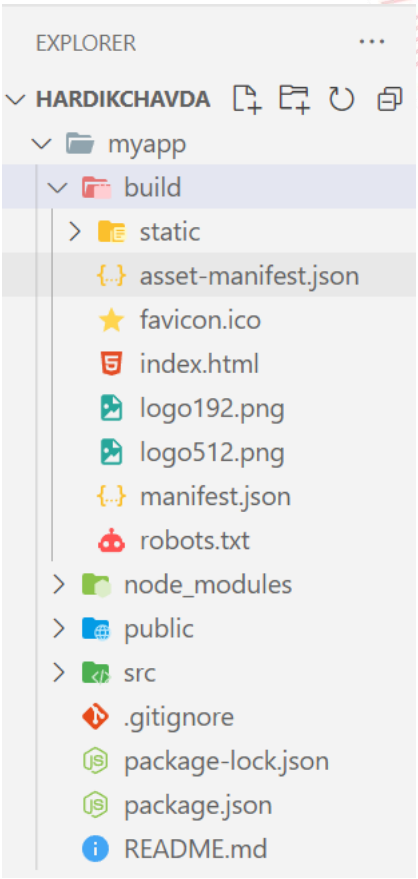
package-lock.json: It is generated automatically for any operations where npm package modifies either the node_modules tree or package.json. It cannot be published. It will be ignored if it finds any other place rather than the top-level package.

package.json: It holds various metadata required for the project. It gives information to npm, which allows to identify the project as well as handle the project's dependencies.

README.md: It provides the documentation to read about React topics.

Next, if we want to make the project for the production mode, type the following command. This command will generate the production build, which is best optimized.

```
D:\HardikChavda\myapp>npm run build
```



For the project to build, these files must exist with exact filenames:

public/index.html is the page template;
src/index.js is the JavaScript entry point.
You can delete or rename the other files.

You may create subdirectories inside src. For faster rebuilds, only files inside src are processed by **webpack**. You need to put any JS and CSS files inside src, otherwise webpack won't see them.

You can, however, create more top-level directories. They will not be included in the production build so you can use them for things like documentation.

Components:

Earlier, the developers write more than thousands of lines of code for developing a single page application. These applications follow the traditional DOM structure, and making changes in them was a very challenging task. If any mistake found, it manually searches the entire application and update accordingly. The component-based approach was introduced to overcome an issue. In this approach, the entire application is divided into a small logical group of code, which is known as components.

Every React component have their own structure, methods as well as APIs. They can be reusable as per your need. For better understanding, consider the entire UI as a tree. Here, the root is the starting component, and each of the other pieces becomes branches, which are further divided into sub-branches.

Creating components

A Component is considered as the core building blocks of a React application. It makes the task of building UIs much easier. Each component exists in the same space, but they work independently from one another and merge all in a parent component, which will be the final UI of your application.

Basic components,

Higher order components are used when you want to share logic across several components regardless of how different they render.

```
import logo from './logo.svg';
import './App.css';

function App() { //Component
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Edit <code>src/App.js</code> and save to reload.
        </p>
        <a
          className="App-link"
          href="https://reactjs.org"
          target="_blank"
          rel="noopener noreferrer"
        >
          Learn React
        </a>
      </header>
    </div>
  );
}
export default App;
```

Given the following file: index.JS

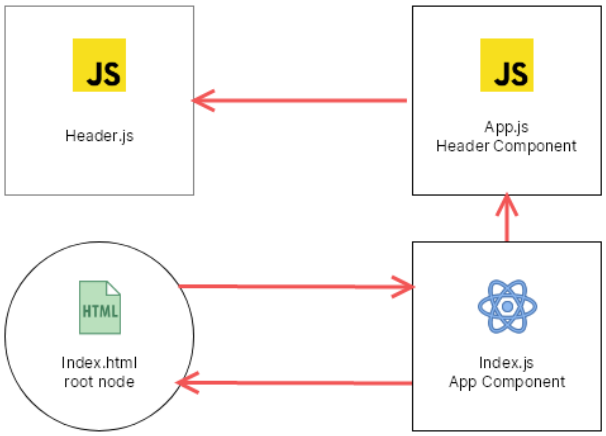
```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';  //Getting Component

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
   <App /> //Using Component
);
```

Nesting components,

A lot of the power of ReactJS is its ability to allow nesting of components. Take the following two components.

Header.js is a component.

```
function Header() {
   return (
      <Header>This is Header</Header>
   )
}
export default Header
```



Header is imported and added in App.js

```
import Header from "./Header";
function App() {
  return (
    <Header />
  );
}
export default App;
```

App.js is imported into index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
   <App />
);
```

Functional Component:

In React, function components are a way to write components that only contain a render method and don't have their own state. They are simply JavaScript functions that may or may not receive data as parameters. We can create a function that takes props(properties) as input and returns what should be rendered. A valid functional component can be shown in the below example.

```
function WelcomeMessage() {
   return <h1>Welcome to the New Component</h1>;
}
```

The functional component is also known as a stateless component because they do not hold or manage state. It can be explained in the below example.

Class Component

Class components are more complex than functional components. It requires you to extend from React. Component and create a render function which returns a React element. You can pass data from one class to other class components. You can create a class by defining a class that extends Component and has a render function. Valid class component is shown in the below example.

```
class MyComponent extends React.Component {
   render() {
      return <div>This is main component. </div>
   }
}
```

The class component is also known as a stateful component because they can hold or manage local state. It can be explained in the below example.

In this example, we are creating the list of unordered elements, where we will dynamically insert StudentName for every object from the data array. Here, we are using ES6 arrow syntax (=>) which looks much cleaner than the old JavaScript syntax. It helps us to create our elements with fewer lines of code. It is especially useful when we need to create a list with a lot of items.

```
import React, { Component } from 'react';
class App extends React.Component {
   constructor() {
      super();
      this.state = {
         data:
            [{ "name": "Abhishek" }, { "name": "Saharsh" }, { "name": "Ajay" }]
      }
   }
   render() {
      return (
         <div>
            <StudentName />
            <ul>
               {this.state.data.map((item) => <List data={item} />)}
```

```
            </ul>
          </div>
        );
    }
}
class StudentName extends React.Component {
    render() {
        return (
            <div>
                <h1>Student Name Detail</h1>
            </div>
        );
    }
}
class List extends React.Component {
    render() {
        return (
            <ul>
                <li>{this.props.data.name}</li>
            </ul>
        );
    }
}
export default App;
```

Introduction to JSX:

As we have already seen that, all of the React components have a render function. The render function specifies the HTML output of a React component. JSX (JavaScript Extension), is a React extension which allows writing JavaScript code that looks like HTML. In other words, JSX is an HTML-like syntax used by React that extends **ECMAScript** so that HTML-like syntax can co-exist with JavaScript/React code. The syntax is used by preprocessors (i.e., transpilers like babel) to transform HTML-like syntax into standard JavaScript objects that a JavaScript engine will parse.

JSX provides you to write HTML/XML-like structures (e.g., DOM-like tree structures) in the same file where you write JavaScript code, then preprocessor will transform these expressions into actual JavaScript code. Just like XML/HTML, JSX tags have a tag name, attributes, and children.

Here, we will write JSX syntax in JSX file and see the corresponding JavaScript code which transforms by preprocessor(**babel**).

JSX File

```
<div>Hello Geetanjali</div>
```

Corresponding Output

```
React.createElement("div", null, "Hello Geetanjali");
```

The above line creates a react element and passing three arguments inside where the first is the name of the element which is div, second is the attributes passed in the div tag, and last is the content you pass which is the "Hello Geetanjali."

Why use JSX?

It is faster than regular JavaScript because it performs optimization while translating the code to JavaScript.
Instead of separating technologies by putting markup and logic in separate files, React uses components that contain both. We will learn components in a further section.
It is type-safe, and most of the errors can be found at compilation time.
It makes easier to create templates.

Nested Elements in JSX

To use more than one element, you need to wrap it with one container element. Here, we use div as a container element which has three nested elements inside it.

```
import React, { Component } from 'react';
class App extends Component {
  render() {
    return (
      <div>
        <h1>Geetanjali</h1>
        <h2>Hardik Chavda</h2>
      </div>
    );
```

```
  }
}
export default App;
```

JSX Attributes

JSX use attributes with the HTML elements same as regular HTML. JSX uses camelcase naming convention for attributes rather than standard naming convention of HTML such as a class in HTML becomes className in JSX because the class is the reserved keyword in JavaScript. We can also use our own custom attributes in JSX. For custom attributes, we need to use data- prefix. In the below example, we have used a custom attribute data-demoAttribute as an attribute for the <p> tag.

In JSX, we can specify attribute values in two ways:

1. As String Literals: We can specify the values of attributes in double quotes:
var element = <h2 className = "firstAttribute">Hello Geetanjali</h2>;
Example

```
import React, { Component } from 'react';
class App extends Component {
  render() {
    return (
      <div>
        <h1 className="hello" >Geetanjali</h1>
        <p data-demoAttribute="demo">Hardik Chavda </p>
      </div>
    );
  }
}
export default App;
```

Output:
Geetanjali
Hardik Chavda

2. As Expressions: We can specify the values of attributes as expressions using curly braces {}:
var element = <h2 className = {varName}>Hello Geetanjali</h2>;
Example

```
import React, { Component } from 'react';
class App extends Component {
  render() {
    return (
      <div>
        <h1 className="hello" >{25 + 20}</h1>
      </div>
    );
  }
}
```

JSX Comments

JSX allows us to use comments that begin with /* and ends with */ and wrapping them in curly braces {} just like in the case of JSX expressions. Below example shows how to use comments in JSX.

Example

```
import React, { Component } from 'react';
class App extends Component {
   render() {
      return (
         <div>
            <h1 className="hello" >Hello Geetanjali</h1>
            {/* This is a comment in JSX */}
         </div>
      );
   }
}
export default App;
```

JSX Styling

React always recommends to use inline styles. To set inline styles, you need to use camelCase syntax. React automatically allows appending px after the number value on specific elements. The following example shows how to use styling in the element.

Example

```
import React, { Component } from 'react';
class App extends Component {
   render() {
      var myStyle = {
         fontSize: 80,
         fontFamily: 'Courier',
         color: '#003300'
      }
      return (
         <div>
            <h1 style={myStyle}>www.geetanjali.com</h1>
         </div>
      );
   }
}
export default App
```

ReactJS JSX

NOTE: JSX cannot allow to use if-else statements. Instead of it, you can use conditional (ternary) expressions. It can be seen in the following example.

Example

```
import React, { Component } from 'react';
class App extends Component {
   render() {
      var i = 5;
      return (
         <div>
            <h1>{i == 1 ? 'True!' : 'False!'}</h1>
         </div>
      );
   }
}
export default App;
```

Props: ReactJS Props,
Props stand for "Properties." They are read-only components. It is an object which stores the value of attributes of a tag and work similar to the HTML attributes. It gives a way to pass data from one component to other components. It is similar to function arguments. Props are passed to the component in the same way as arguments passed in a function.

Props are immutable so we cannot modify the props from inside the component. Inside the components, we can add attributes called props. These attributes are available in the component as this.props and can be used to render dynamic data in our render method.

When you need immutable data in the component, you have to add props to reactDom.render() method in the main.js file of your ReactJS project and used it inside the component in which you need.

App.js

```
import React, { Component } from 'react';
class App extends React.Component {
   render() {
      return (
         <div>
            <h1> Welcome to {this.props.name} </h1>
            <p> <h4> Geetanjali is one of the best College in India. </h4> </p>
         </div>
      );
   }
}
export default App;
```

Main.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App.js';

ReactDOM.render(
         <App name="Geetanjali!!" />, document.getElementById('app')
);
```

Default Props

It is not necessary to always add props in the reactDom.render() element. You can also set default props directly on the component constructor.

App.js

```
import React, { Component } from 'react';
class App extends React.Component {
   render() {
      return (
         <div>
            <h1>Default Props Example</h1>
            <h3>Welcome to {this.props.name}</h3>
            <p>It is one of the best College in India.</p>
         </div>
      );
   }
}
App.defaultProps = {
   name: "Geetanjali"
}
export default App;
```

Main.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App.js';

ReactDOM.render(<App />, document.getElementById('app'));
```

React State,
Components, we got to know that React Components can be broadly classified into Functional and Class Components. It is also seen that Functional Components are faster and much simpler than Class Components. The primary difference between the two is the availability of the State.

What is State?
The state is an instance of React Component Class can be defined as an object of a set of observable properties that control the behavior of the component. In other words, the State of a component is an object that holds some information that may change over the lifetime of the component. For example, let us think of the clock that we created in this article, we were calling the render() method every second explicitly, but React provides a better way to achieve the same result and that is by using State, storing the value of time as a member of the component's state. We will look into this more elaborately later in the article.

Difference of Props and State.
We have already learned about Props and we got to know that Props are also objects that hold information to control the behavior of that particular component, sounds familiar to State indeed but props and states are nowhere near be same. Let us differentiate the two.
Props are immutable i.e. once set the props cannot be changed, while State is an observable object that is to be used to hold data that may change over time and to control the behavior after each change.

States can be used in Class Components, Functional components with the use of React Hooks (useState and other methods) while Props don't have this limitation.

While Props are set by the parent component, State is generally updated by event handlers. It can be implemented using State where the probable values of the State can be either light or dark and upon selection, the IDE changes its color.

Now we have learned the basics of State and are able to differentiate it from Props. We have also seen a few places where we can use State now all that is left is to know about the basic conventions of using the React State before implementing one for ourselves.

Conventions of Using State in React:

State of a component should prevail throughout the lifetime, thus we must first have some initial state, to do so we should define the State in the constructor of the component's class. To define a state of any Class we can use the sample format below.

```
class MyClass extends React.Component
{
   constructor(props){
      super(props);
      this.state = { attribute: "value" };
   }
}
```

State should never be updated explicitly. React uses an observable object as the state that observes what changes are made to the state and helps the component behave accordingly. For example, if we update the state of any component like the following the webpage will not re-render itself because React State will not be able to detect the changes made.

```
this.state.attribute = "new-value";
```

Thus, React provides its own method setState(). setState() method takes a single parameter and expects an object which should contain the set of values to be updated. Once the update is done the method implicitly calls the render() method to repaint the page. Hence, the correct method of updating the value of a state will be similar to the code below.

```
this.setState({attribute: "new-value"});
```

The only time we are allowed to define the state explicitly is in the constructor to provide the initial state.

React is highly efficient and thus uses asynchronous state updates i.e. React may update multiple setState() updates in a single go. Thus using the value of the current state may not always generate the desired result. For example, let us take a case where we must keep a count (Likes of a Post). Many developers may miswrite the code as below.

```
this.setState({counter: this.state.count + this.props.diff});
```

Now due to asynchronous processing, this.state.count may produce an undesirable result. A more appropriate approach would be to use the following.

```
this.setState((prevState, props) => ({
        counter: prevState.count + props.diff
}));
```

In the above code we are using the ES6 thick arrow function format to take the previous state and props of the component as parameters and are updating the counter. The same can be written using the default functional way as follows.

```
this.setState(function(prevState, props){
        return {counter: prevState.count + props.diff};
});
```

State updates are independent. The state object of a component may contain multiple attributes and React allows to use setState() function to update only a subset of those attributes as well as using multiple setState() methods to update each attribute value independently. For example, let us take the following component state into account.

```
this.state = {
        darkTheme: False,
        searchTerm: ''};
```

The above definition has two attributes we can use a single setState() method to update both together, or we can use separate setState() methods to update the attributes independently. React internally merges setState() methods or updates only those attributes which are needed.

Destructing Props and State,
Destructuring is a simple property that is used to make code much clear and readable, mainly when we pass props in React.

What is Destructuring?

- Destructuring is a characteristic of JavaScript, It is used to take out sections of data from an array or objects, We can assign them to new own variables created by the developer.
- In destructuring, It does not change an array or any object, it makes a copy of the desired object or array element by assigning them in its own new variables, later we can use this new variable in React (class or functional) components.
- It makes the code more clear. When we access the props using this keyword, we have to use this/ this.props throughout the program, but by the use of restructuring, we can discard this/ this.props by assigning them in new variables.
- This is very difficult to monitor props in complex applications, so by assigning these props in new own variables we can make a code more readable.

Advantages of Destructuring:

- It makes developer's life easy, by assigning their own variables.
- Nested data is more complex, it takes time to access, but by the use of destructuring, we can access faster of nested data.
- It improves the sustainability, readability of code.
- It helps to cut the amount of code used in an application.
- It trims the number of steps taken to access data properties.
- It provides components with the exact data properties.
- It saves time from iterate over an array of objects multiple times.
- In ReactJS We use multiple times ternary operators inside the render function, without destructuring it looks complex and hard to access them, but by the use of destructuring, we can improve the readability of ternary operators.

How to use Destructuring?

We can use the Destructuring in the following method in ReactJS:

In this example, we are going to simply display some words using destructuring and without destructuring.

App.js:

```
import React from "react"
import Greet from './component/Greet'
```

```
class App extends React.component {
   render() {
      return (
         <div className="App">
            <Greet active="Hardik Chavda" activeStatus="CSE" />
         </div>
      );
   }
}
export default App;
```

Without Destructuring:

```
import React from 'react';

const Greet = props => {
   return (
      <div>
         <div className="XYZ">
            <h3> {props.active} </h3>
         </div>

         <div className="PQR">
            <h1>{props.activeStatus}</h1>
         </div>
      </div>
   )
}
export default Greet;
```

With Destructuring:

```
import React from 'react';

const Greet = () => {
   const {active, activeStatus } = props
   return (
      <div>
         <div >
            <h3> {active} </h3>
         </div>

         <div >
            <h1>{activeStatus}</h1>
         </div>
      </div>
   )
}
export default Greet;
```

setState,

All the React components can have a state associated with them. The state of a component can change either due to a response to an action performed by the user or an event triggered by the system. Whenever the state changes, React re-renders the component to the browser. Before updating the value of the state, we need to build an initial state setup. Once we are done with it, we use the setState() method to change the state object. It ensures that the component has been updated and calls for re-rendering of the component.

setState is asynchronous call means if synchronous call gets called it may not get updated at right time like to know current value of object after update using setState it may not get give current updated value on console. To get some behavior of synchronous need to pass function instead of object to setState.

Syntax: We can use setState() to change the state of the component directly as well as through an arrow function.

```
setState({ stateName: updatedStateValue })

// OR
setState((prevState) => ({
  stateName: prevState.stateName + 1
}))
```

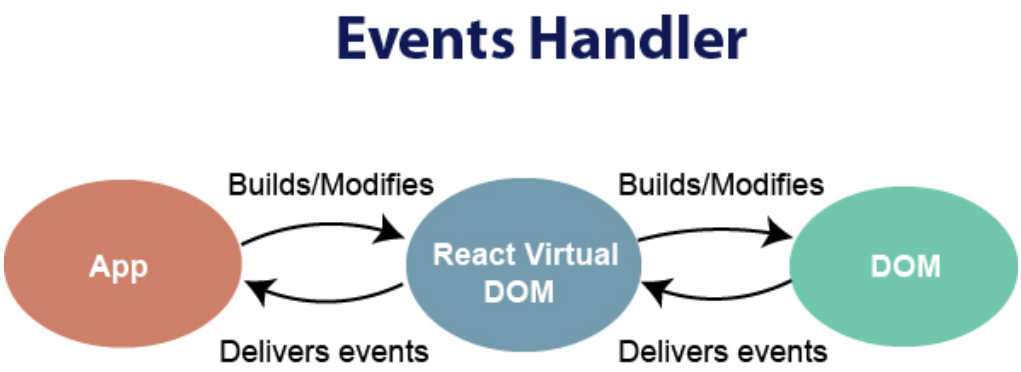
Example

```
import React, { Component } from 'react'

class App extends Component {
  constructor(props) {
    super(props)

    // Set initial state
    this.state = {
      greeting:
        'Click the button to receive greetings'
    }

    // Binding this keyword
    this.updateState = this.updateState.bind(this)
  }

  updateState() {
    // Changing state
    this.setState({
      greeting:
        'HardikChavda welcomes you !!'
    })
  }
```

```
  render() {
    return (
      <div>
        <h2>Greetings Portal</h2>
        <p>{this.state.greeting}</p>

        {/* Set click handler */}
        <button onClick={this.updateState}>
          Click me!
        </button>
      </div>
    )
  }
}

export default App;
```

Event Handling: Event Handling and Binding event handlers,
An event is an action that could be triggered as a result of the user action or system generated event. For example, a mouse click, loading of a web page, pressing a key, window resizing, and other interactions are called events.

React has its own event handling system which is very similar to handling events on DOM elements. The react event handling system is known as Synthetic Events. The synthetic event is a cross-browser wrapper of the browser's native event.



Handling events with react have some syntactic differences from handling events on DOM. These are:

React events are named as camelCase instead of lowercase.
With JSX, a function is passed as the event handler instead of a string. For example:

Event declaration in plain HTML:

```
<button onclick="showMessage()">
Hello Geetanjali
</button>
```

Event declaration in React:

```
<button onClick={showMessage}>
Hello Geetanjali
</button>
```

In react, we cannot return false to prevent the default behavior. We must call preventDefault event explicitly to prevent the default behavior. For example:

In plain HTML, to prevent the default link behavior of opening a new page, we can write:

```
<a href="#" onClick="console.log('You had clicked a Link.'); return false">
Click_Me
</a>
```

In React, we can write it as:

```
function ActionLink() {
  function handleClick(e) {
    e.preventDefault();
    console.log('You had clicked a Link.');
  }
  return (
    <a href="#" onClick={handleClick}>
      Click_Me
    </a>
  );
}
```

In the above example, e is a Synthetic Event which defines according to the W3C spec.

Now let us see how to use Events in React.

Example

In the below example, we have used only one component and added an onChange event. This event will trigger the changeText function, which returns the company name.

```
import React, { Component } from 'react';
class App extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      companyName: ''
    };
  }
  changeText(event) {
    this.setState({
      companyName: event.target.value
    });
  }
  render() {
    return (
      <div>
        <h2>Simple Event Example</h2>
        <label htmlFor="name">Enter company name: </label>
        <input type="text" id="companyName" onChange={this.changeText.bind(this)} />
        <h4>You entered: {this.state.companyName}</h4>
      </div>
    );
  }
}
export default App;
```

Rendering: Conditional Rendering and List Rendering,
In React, we can create multiple components which encapsulate behavior that we need. After that, we can render them depending on some conditions or the state of our application. In other words, based on one or several conditions, a component decides which elements it will return. In React, conditional rendering works the same way as the conditions work in JavaScript. We use JavaScript operators to create elements representing the current state, and then React Components update the UI to match them.

From the given scenario, we can understand how conditional rendering works. Consider an example of handling a login/logout button. The login and logout buttons will be separate components. If a user logged in, render the logout component to display the logout button. If a user is not logged in, render the login component to display the login button. In React, this situation is called conditional rendering.

There is more than one way to do conditional rendering in React. They are given below.

- **if**
- **ternary operator**
- **logical && operator**
- **switch case operator**
- **Conditional Rendering with enums**

With if
It is the easiest way to have a conditional rendering in React in the render method. It is restricted to the total block of the component. IF the condition is true, it will return the element to be rendered. It can be understood in the below example.

Example

```
function UserLoggin(props) {
   return <h1>Welcome back!</h1>;
}
function GuestLoggin(props) {
   return <h1>Please sign up.</h1>;
}
function SignUp(props) {
   const isLoggedIn = props.isLoggedIn;
   if (isLoggedIn) {
      return <UserLogin />;
   }
   return <GuestLogin />;
}

ReactDOM.render(
   <SignUp isLoggedIn={false} />,
   document.getElementById('root')
);
```

Logical && operator

This operator is used for checking the condition. If the condition is true, it will return the element right after &&, and if it is false, React will ignore and skip it.

Syntax

```
{
    condition &&
    // whatever written after && will be a part of output.
}
```

If you run the below code, you will not see the alert message because the condition is not matching.

```
('geetanjali' == 'Geetanjali') && alert('This alert will never be shown!')
```

If you run the below code, you will see the alert message because the condition is matching.

```
(10 > 5) && alert('This alert will be shown!')
```

Example

```
import React from 'react';
function Example() {
  return (<div>
    {
      (10 > 5) && alert('This alert will be shown!')
    }
  </div>
  );
}
```


You can see in the above output that as the condition (10 > 5) evaluates to true, the alert message is successfully rendered on the screen.

Ternary operator

The ternary operator is used in cases where two blocks alternate given a certain condition. This operator makes your if-else statement more concise. It takes three operands and is used as a shortcut for the if statement.

Syntax

```
condition ?  true : false
```

If the condition is true, statement1 will be rendered. Otherwise, false will be rendered.

Example

```
render() {
   const isLoggedIn = this.state.isLoggedIn;
   return (
      <div>
         Welcome {isLoggedIn ? 'Back' : 'Please login first'}.
      </div>
   );
}
```

Switch case operator

Sometimes it is possible to have multiple conditional renderings. In the switch case, conditional rendering is applied based on a different state.

Example

```
function NotificationMsg({ text }) {
   switch (text) {
      case 'Hi All':
         return <Message: text={text} />;
      case 'Hello Geetanjali':
         return <Message text={text} />;
      default:
         return null;
   }
}
```

Conditional Rendering with enums

An enum is a great way to have multiple conditional rendering. It is more readable as compared to switch case operators. It is perfect for mapping between different state. It is also perfect for mapping in more than one condition. It can be understood in the below example.

Example

```
function NotificationMsg({ text, state }) {
   return (
      <div>
         {{
            info: <Message text={text} />,
            warning: <Message text={text} />,
         }[state]}
      </div>
   );
}
```

Conditional Rendering Example

In the below example, we have created a stateful component called App which maintains the login control. Here, we create three components representing Logout, Login, and Message component. The stateful component App will render either or depending on its current state.

```
import React, { Component } from 'react';
// Message Component
function Message(props) {
   if (props.isLoggedIn)
      return <h1>Welcome Back!!!</h1>;
   else
      return <h1>Please Login First!!!</h1>;
}
// Login Component
function Login(props) {
   return (
      <button onClick={props.clickInfo}> Login </button>
   );
}
// Logout Component
function Logout(props) {
   return (
      <button onClick={props.clickInfo}> Logout </button>
   );
}
class App extends Component {
   constructor(props) {
      super(props);
      this.handleLogin = this.handleLogin.bind(this);
      this.handleLogout = this.handleLogout.bind(this);
      this.state = { isLoggedIn: false };
   }
   handleLogin() {
      this.setState({ isLoggedIn: true });
   }
   handleLogout() {
      this.setState({ isLoggedIn: false });
   }
   render() {
      return (
         <div>
            <h1> Conditional Rendering Example </h1>
            <Message isLoggedIn={this.state.isLoggedIn} />
            {
               (this.state.isLoggedIn) ? (
                  <Logout clickInfo={this.handleLogout} />
```

```
            ) : (
               <Login clickInfo={this.handleLogin} />
            )
         }
      </div>
    );
  }
}
export default App;
```

List and keys,

Lists are used to display data in an ordered format and mainly used to display menus on websites. In React, Lists can be created in a similar way as we create lists in JavaScript. Let us see how we transform Lists in regular JavaScript.

The map() function is used for traversing the lists. In the below example, the map() function takes an array of numbers and multiplies their values with 5. We assign the new array returned by map() to the variable multiplyNums and log it.

Example

```
var numbers = [1, 2, 3, 4, 5];
const multiplyNums = numbers.map((number) => {
   return (number * 5);
});
console.log(multiplyNums);
OP:
[5, 10, 15, 20, 25]
```


Now, let us see how we create a list in React. To do this, we will use the map() function for traversing the list element, and for updates, we enclosed them between curly braces {}. Finally, we assign the array elements to listItems. Now, include this new list inside <ul> elements and render it to the DOM.

Example

```
import React from 'react';
import ReactDOM from 'react-dom';
const myList = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
const listItems = myList.map((myList) => {
   return <li>{myList}</li>;
});
ReactDOM.render(
   <ul> {listItems} </ul>,
   document.getElementById('root')
);
```

React Lists

Rendering Lists inside components

In the previous example, we had directly rendered the list to the DOM. But it is not a good practice to render lists in React. In React, we had already seen that everything is built as individual components. Hence, we would need to render lists inside a component. We can understand it in the following code.

Example

```
import React from 'react';
import ReactDOM from 'react-dom';

function NameList(props) {
   const myLists = props.myLists;
   const listItems = myLists.map((myList) =>
      <li>{myList}</li>
   );
   return (
      <div>
         <h2>Rendering Lists inside component</h2>
         <ul>{listItems}</ul>
      </div>
   );
}
const myLists = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
ReactDOM.render(
   <NameList myLists={myLists} />,
   document.getElementById('app')
);
export default App;
```


Index as Key Anti-pattern

A key is a unique identifier. In React, it is used to identify which items have changed, updated, or deleted from the Lists. It is useful when we dynamically create components or when the users alter the lists. It also helps to determine which components in a collection need to be re-rendered instead of re-rendering the entire set of components every time.

Keys should be given inside the array to give the elements a stable identity. The best way to pick a key as a string that uniquely identifies the items in the list. It can be understood with the below example.

Example

```
const stringLists = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
const updatedLists = stringLists.map((strList) => {
   <li key={strList.id}> {strList} </li>;
});
```

If there are no stable IDs for rendered items, you can assign the item index as a key to the lists. It can be shown in the below example.

Example

```
const stringLists = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
const updatedLists = stringLists.map((strList, index) => {
   <li key={index}> {strList} </li>;
});
```

Note: It is not recommended to use indexes for keys if the order of the item may change in future. It creates confusion for the developer and may cause issues with the component state.

Using Keys with component

Consider you have created a separate component for ListItem and extracting ListItem from that component. In this case, you should have to assign keys on the <ListItem /> elements in the array, not to the <li> elements in the ListItem itself. To avoid mistakes, you have to keep in mind that keys only make sense in the context of the surrounding array. So, anything you are returning from map() function is recommended to be assigned a key.

Example: Incorrect Key usage

```
import React from 'react';
import ReactDOM from 'react-dom';

function ListItem(props) {
   const item = props.item;
   return (
      // Wrong! No need to specify the key here.
      <li key={item.toString()}>
         {item}
      </li>
   );
}
function NameList(props) {
   const myLists = props.myLists;
   const listItems = myLists.map((strLists) =>
      // The key should have been specified here.
      <ListItem item={strLists} />
   );
   return (
      <div>
         <h2>Incorrect Key Usage Example</h2>
         <ol>{listItems}</ol>
      </div>
   );
}
```

```
const myLists = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
ReactDOM.render(
   <NameList myLists={myLists} />,
   document.getElementById('app')
);
export default App;
```

In the given example, the list is rendered successfully. But it is not a good practice that we had not assigned a key to the map() iterator.

React Keys

Example: Correct Key usage

To correct the above example, we should have to assign a key to the map() iterator.

```
import React from 'react';
import ReactDOM from 'react-dom';

function ListItem(props) {
   const item = props.item;
   return (
      // No need to specify the key here.
      <li> {item} </li>
   );
}
function NameList(props) {
   const myLists = props.myLists;
   const listItems = myLists.map((strLists) =>
      // The key should have been specified here.
      <ListItem key={myLists.toString()} item={strLists} />
   );
   return (
      <div>
         <h2>Correct Key Usage Example</h2>
         <ol>{listItems}</ol>
      </div>
   );
}
const myLists = ['Peter', 'Sachin', 'Kevin', 'Dhoni', 'Alisa'];
ReactDOM.render(
   <NameList myLists={myLists} />,
   document.getElementById('app')
);
export default App;
```

Uniqueness of Keys among Siblings

We had discussed that keys assignment in arrays must be unique among their siblings. However, it doesn't mean that the keys should be globally unique. We can use the same set of keys in producing two different arrays. It can be understood in the below example.

Example

```
import React from 'react';
import ReactDOM from 'react-dom';
function MenuBlog(props) {
  const titlebar = (
    <ol>
      {props.data.map((show) =>
        <li key={show.id}>
          {show.title}
        </li>
      )}
    </ol>
  );
  const content = props.data.map((show) =>
    <div key={show.id}>
      <h3>{show.title}: {show.content}</h3>
    </div>
  );
  return (
    <div>
      {titlebar}
      <hr />
      {content}
    </div>
  );
}
const data = [
  { id: 1, title: 'First', content: 'Welcome to Geetanjali!!' },
  { id: 2, title: 'Second', content: 'It is the best ReactJS Tutorial!!' },
  { id: 3, title: 'Third', content: 'Here, you can learn all the ReactJS topics!!' }
];
ReactDOM.render(
  <MenuBlog data={data} />,
  document.getElementById('app')
);
export default App;
```

Introduction: Basic form handling

Forms are an integral part of any modern web application. It allows the users to interact with the application as well as gather information from the users. Forms can perform many tasks that depend on the nature of your business requirements and logic such as authentication of the user, adding user, searching, filtering, booking, ordering, etc. A form can contain text fields, buttons, checkbox, radio button, etc.

Creating Form

React offers a stateful, reactive approach to build a form. The component rather than the DOM usually handles the React form. In React, the form is usually implemented by using controlled components.

There are mainly two types of form input in React.

- Uncontrolled component
- Controlled component

Uncontrolled component

The uncontrolled input is similar to the traditional HTML form inputs. The DOM itself handles the form data. Here, the HTML elements maintain their own state that will be updated when the input value changes. To write an uncontrolled component, you need to use a **ref** to get form values from the DOM. In other words, there is no need to write an event handler for every state update. You can use a ref to access the input field value of the form from the DOM.

Example

In this example, the code accepts a field username and company name in an uncontrolled component.

```
import React, { Component } from 'react';
class App extends React.Component {
   constructor(props) {
      super(props);
      this.updateSubmit = this.updateSubmit.bind(this);
      this.input = React.createRef();
   }
   updateSubmit(event) {
      alert('You have entered the UserName and CompanyName successfully.');
      event.preventDefault();
   }
   render() {
      return (
         <form onSubmit={this.updateSubmit}>
            <h1>Uncontrolled Form Example</h1>
            <label>Name:
               <input type="text" ref={this.input} />
            </label>
```

```
            <label>
              CompanyName:
              <input type="text" ref={this.input} />
            </label>
            <input type="submit" value="Submit" />
          </form>
      );
    }
}
export default App;
```

After filling the data in the field, you get the message that can be seen in the below screen.

Controlled Component

In HTML, form elements typically maintain their own state and update it according to the user input. In the controlled component, the input form element is handled by the component rather than the DOM. Here, the mutable state is kept in the state property and will be updated only with the setState() method.

Controlled components have functions that govern the data passing into them on every onChange event, rather than grabbing the data only once, e.g., when you click a submit button. This data is then saved to state and updated with the setState() method. This makes component have better control over the form elements and data.

A controlled component takes its current value through props and notifies the changes through callbacks like an onChange event. A parent component "controls" this change by handling the callback and managing its own state and then passing the new values as props to the controlled component. It is also called as a "dumb component."

Example

```
import React, { Component } from 'react';
class App extends React.Component {
   constructor(props) {
      super(props);
      this.state = { value: '' };
      this.handleChange = this.handleChange.bind(this);
      this.handleSubmit = this.handleSubmit.bind(this);
   }
   handleChange(event) {
      this.setState({ value: event.target.value });
   }
   handleSubmit(event) {
      alert('You have submitted the input successfully: ' + this.state.value);
      event.preventDefault();
   }
   render() {
```

```
    return (
      <form onSubmit={this.handleSubmit}>
        <h1>Controlled Form Example</h1>
        <label>
          Name:
          <input type="text" value={this.state.value} onChange={this.handleChange} />
        </label>
        <input type="submit" value="Submit" />
      </form>
    );
  }
}
export default App;
```

Handling Multiple Inputs in Controlled Component

If you want to handle multiple controlled input elements, add a name attribute to each element, and then the handler function decided what to do based on the value of event.target.name.

Example

```
import React, { Component } from 'react';
class App extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      personGoing: true,
      numberOfPersons: 5
    };
    this.handleInputChange = this.handleInputChange.bind(this);
  }
  handleInputChange(event) {
    const target = event.target;
    const value = target.type === 'checkbox' ? target.checked : target.value;
    const name = target.name;
    this.setState({
      [name]: value
    });
  }
  render() {
    return (
      <form>
        <h1>Multiple Input Controlled Form Example</h1>
        <label>
          Is Person going:
          <input
```

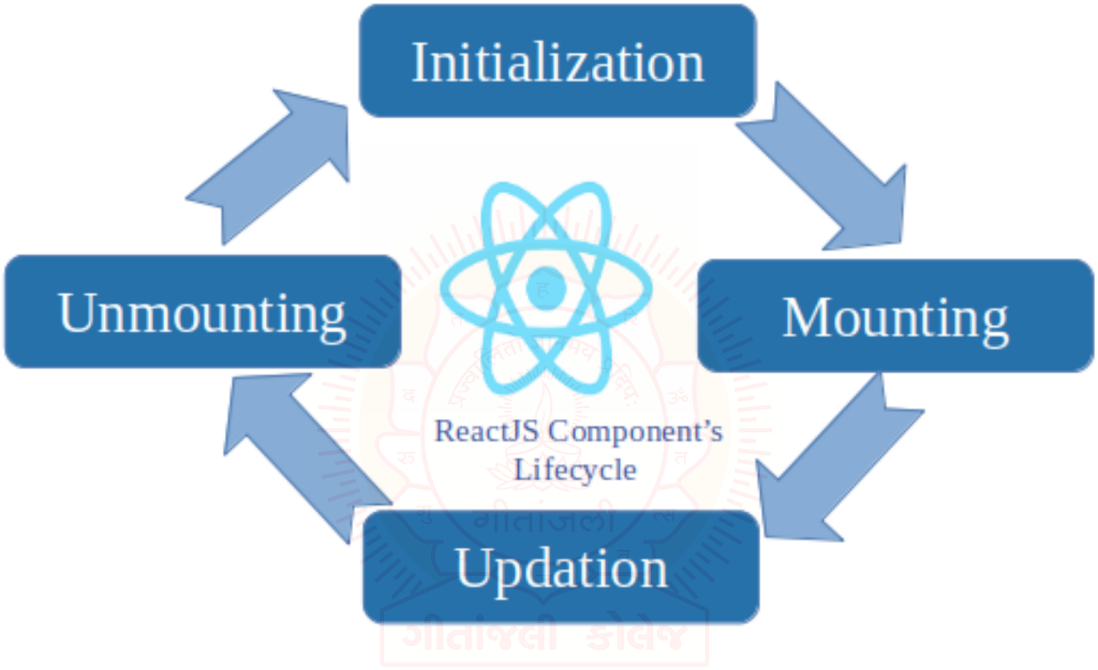
```
                name="personGoing"
                type="checkbox"
                checked={this.state.personGoing}
                onChange={this.handleInputChange} />
            </label>
            <br />
            <label>
              Number of persons:
              <input
                name="numberOfPersons"
                type="number"
                value={this.state.numberOfPersons}
                onChange={this.handleInputChange} />
            </label>
          </form>
      );
    }
}
export default App;
```

Difference table between **controlled** and **uncontrolled component**.

Sr. No	Controlled	Uncontrolled
1	It does not maintain its internal state.	It maintains its internal states.
2	Here, data is controlled by the parent component.	Here, data is controlled by the DOM itself.
3	It accepts its current value as a prop.	It uses a ref for their current values.
4	It allows validation control.	It does not allow validation control.
5	It has better control over the form elements and data.	It has limited control over the form elements and data.

Components: Components Life Cycle Methods,
In ReactJS, every component creation process involves various lifecycle methods. These lifecycle methods are termed as component's lifecycle. These lifecycle methods are not very complicated and are called at various points during a component's life. The lifecycle of the component is divided into four phases. They are:

- Initial Phase
- Mounting Phase
- Updating Phase
- Unmounting Phase



1. Initial Phase
It is the birth phase of the lifecycle of a ReactJS component. Here, the component starts its journey on a way to the DOM. In this phase, a component contains the default Props and initial State. These default properties are done in the constructor of a component. The initial phase only occurs once and consists of the following methods.

getDefaultProps()
It is used to specify the default value of this.props. It is invoked before the creation of the component or any props from the parent is passed into it.

getInitialState()
It is used to specify the default value of this.state. It is invoked before the creation of the component.

2. Mounting Phase

In this phase, the instance of a component is created and inserted into the DOM. It consists of the following methods.

componentWillMount()

This is invoked immediately before a component gets rendered into the DOM. In the case, when you call setState() inside this method, the component will not re-render.

componentDidMount()

This is invoked immediately after a component gets rendered and placed on the DOM. Now, you can do any DOM querying operations.

render()

This method is defined in each and every component. It is responsible for returning a single root HTML node element. If you don't want to render anything, you can return a null or false value.

3. Updating Phase

It is the next phase of the lifecycle of a react component. Here, we get new Props and change State. This phase also allows to handle user interaction and provide communication with the components hierarchy. The main aim of this phase is to ensure that the component is displaying the latest version of itself. Unlike the Birth or Death phase, this phase repeats again and again. This phase consists of the following methods.

componentWillRecieveProps()

It is invoked when a component receives new props. If you want to update the state in response to prop changes, you should compare this.props and nextProps to perform state transition by using this.setState() method.

shouldComponentUpdate()

It is invoked when a component decides any changes/updation to the DOM. It allows you to control the component's behavior of updating itself. If this method returns true, the component will update. Otherwise, the component will skip the updating.

componentWillUpdate()

It is invoked just before the component updating occurs. Here, you can't change the component state by invoking this.setState() method. It will not be called, if shouldComponentUpdate() returns false.

render()

It is invoked to examine this.props and this.state and return one of the following types: React elements, Arrays and fragments, Booleans or null, String and Number. If shouldComponentUpdate() returns false, the code inside render() will be invoked again to ensure that the component displays itself properly.

componentDidUpdate()

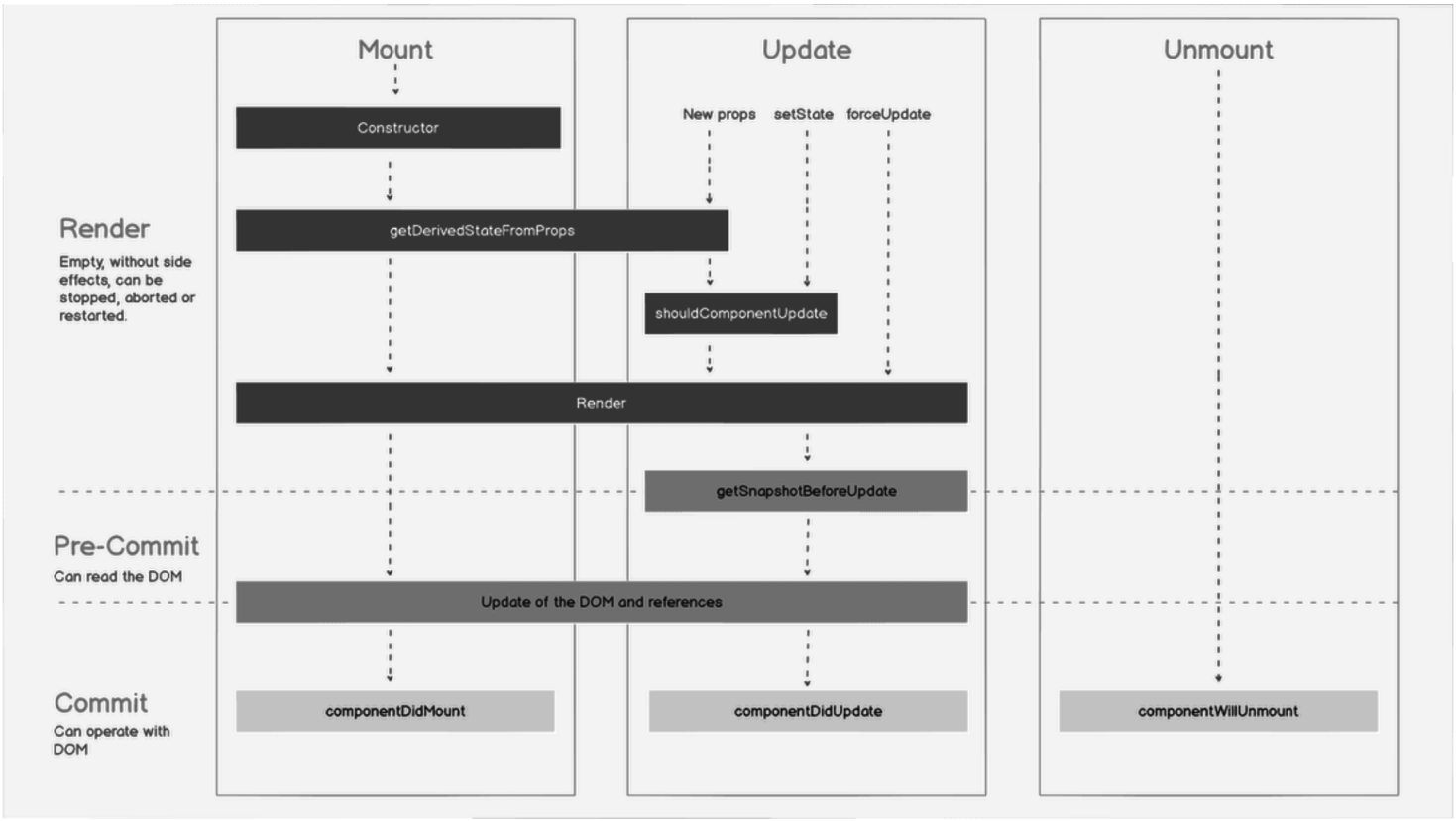
It is invoked immediately after the component updating occurs. In this method, you can put any code inside this which you want to execute once the updating occurs. This method is not invoked for the initial render.

4. Unmounting Phase

It is the final phase of the react component lifecycle. It is called when a component instance is destroyed and unmounted from the DOM. This phase contains only one method and is given below.

componentWillUnmount()

This method is invoked immediately before a component is destroyed and unmounted permanently. It performs any necessary cleanup related task such as invalidating timers, event listeners, canceling network requests, or cleaning up DOM elements. If a component instance is unmounted, you cannot mount it again.



Example

```
import React, { Component } from 'react';

class App extends React.Component {
   constructor(props) {
      super(props);
      this.state = { hello: "Geetanjali" };
      this.changeState = this.changeState.bind(this)
   }
   render() {
      return (
         <div>
            <h1>ReactJS component's Lifecycle</h1>
            <h3>Hello {this.state.hello}</h3>
            <button onClick={this.changeState}>Click Here!</button>
         </div>
      );
   }
   componentWillMount() {
      console.log('Component Will MOUNT!')
   }
   componentDidMount() {
      console.log('Component Did MOUNT!')
   }
   changeState() {
      this.setState({ hello: "All!! - Its a great reactjs tutorial." });
   }
   componentWillReceiveProps(newProps) {
      console.log('Component Will Recieve Props!')
   }
   shouldComponentUpdate(newProps, newState) {
      return true;
   }
   componentWillUpdate(nextProps, nextState) {
      console.log('Component Will UPDATE!');
   }
   componentDidUpdate(prevProps, prevState) {
      console.log('Component Did UPDATE!')
   }
   componentWillUnmount() {
      console.log('Component Will UNMOUNT!')
   }
}
export default App;
```

Pure Components

Generally, In ReactJS, we use shouldComponentUpdate() Lifecycle method to customize the default behavior and implement it when the React component should re-render or update itself.

Prerequisite:

- ReactJS Components
- ReactJS Components – Set 2

Now, ReactJS has provided us with a Pure Component. If we extend a class with Pure Component, there is no need for shouldComponentUpdate() Lifecycle Method. ReactJS Pure Component Class compares current state and props with new props and states to decide whether the React component should re-render itself or Not.

In simple words, If the previous value of state or props and the new value of state or props is the same, the component will not re-render itself. Since Pure Components restricts the re-rendering when there is no use of re-rendering of the component. Pure Components are Class Components which extend React.PureComponent.

Example: Program to demonstrate the creation of Pure Components.

```
import React from 'react';

export default class Test extends React.PureComponent {
   render() {
      return <h1>Welcome to Geetanjali</h1>;
   }
}
```

Extending React Class Components with Pure Components ensures the higher performance of the Component and ultimately makes your application faster, While in the case of Regular Component, it will always re-render either value of State and Props changes or not.

While using Pure Components, Things to be noted are that, In these components, the Value of State and Props are Shallow Compared (Shallow Comparison) and It also takes care of "shouldComponentUpdate" Lifecycle method implicitly.

So there is a possibility that if these State and Props Objects contain nested data structure then Pure Component's implemented shouldComponentUpdate will return false and will not update the whole subtree of Children of this Class Component. So in Pure Component, the nested data structure doesn't work properly.

In this case, State and Props Objects should be simple objects and Child Elements should also be Pure, which means to return the same output for the same input values at any instance.

Fragments

A common pattern in React is for a component to return multiple elements. Fragments let you group a list of children without adding extra nodes to the DOM.

There is also a new short syntax for declaring them.

```
render() {
  return (
    <React.Fragment>
      <ChildA />
      <ChildB />
      <ChildC />
    </React.Fragment>
  );
}
```

Motivation

A common pattern is for a component to return a list of children. Take this example React snippet:

```
class Table extends React.Component {
  render() {
    return (
      <table>
        <tr>
          <Columns />
        </tr>
      </table>
    );
  }
}
```


<Columns /> would need to return multiple <td> elements in order for the rendered HTML to be valid. If a parent div was used inside the render() of <Columns />, then the resulting HTML will be invalid.

```
class Columns extends React.Component {
  render() {
    return (
      <div>
        <td>Hello</td>
        <td>World</td>
      </div>
    );
  }
}
```

results in a < Table /> output of:

```
<table>
  <tr>
    <div>
      <td>Hello</td>
      <td>World</td>
    </div>
  </tr>
</table>
```

Fragments solve this problem.

Usage

```
class Columns extends React.Component {
  render() {
    return (
      <React.Fragment>
        <td>Hello</td>
        <td>World</td>
      </React.Fragment>
    );
  }
}
```

which results in a correct <Table /> output of:

```
<table>
  <tr>
    <td>Hello</td>
    <td>World</td>
  </tr>
</table>
```

Short Syntax

There is a new, shorter syntax you can use for declaring fragments. It looks like empty tags: You can use <></> the same way you'd use any other element except that it doesn't support keys or attributes.

Keyed Fragments

Fragments declared with the explicit <React.Fragment> syntax may have keys. A use case for this is mapping a collection to an array of fragments — for example, to create a description list:

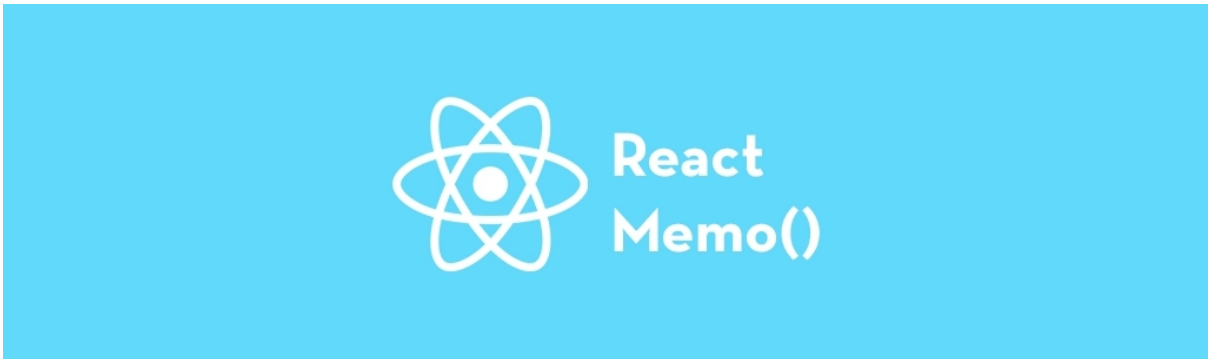
```
class Columns extends React.Component {
  render() {
    return (
      <>
```

```
          <td>Hello</td>
          <td>World</td>
        </>
      );
    }
}
```

```
function Glossary(props) {
  return (
    <dl>
      {props.items.map(item => (
        // Without the `key`, React will fire a key warning
        <React.Fragment key={item.id}>
          <dt>{item.term}</dt>
          <dd>{item.description}</dd>
        </React.Fragment>
      ))}
    </dl>
  );
}
```

key is the only attribute that can be passed to Fragment. In the future, we may add support for additional attributes, such as event handlers.

Memo



What is React Memo?Anchor

Components in React are designed to re-render whenever the state or props value changes. However, this can impact your application's performance because, even if the change is only intended to affect the parent component, every other child component attached to the parent component will re-render. When a parent component re-renders, so do all of its child components.

React Memo is a **higher-order component** that wraps around a component to memoize the rendered output and avoid unnecessary renderings. This improves performance because it memoizes the result and skips rendering to reuse the last rendered result.

There are two ways you can wrap your component with React.memo(). It is either you wrap the actual component directly without having to create a new variable to store the memoized component:

```
const myComponent = React.memo((props) => {
   /* render using props */
});

export default myComponent;
```

Another option is to create a new variable to store the memoized component and then export the new variable:

```
const myComponent = (props) => {
   /* render using props */
};

export const MemoizedComponent = React.memo(myComponent);
```

In the example above, myComponent outputs the same content as MemoizedComponent, but the difference between both is that MemoizedComponent render is memoized. This means that this component will only re-render when the props change.

Note: A memoized component will only re-render when there is a change in props value or when the state and context of the component change.

When to use React MemoAnchor

You now know what it means to memoize a component and the advantages of optimization. But this doesn't mean you should memoize all your components to ensure maximum performance optimization .

It is important to know when and where to memoize your component else it would not fulfill its purpose. For example, React Memo is used to avoid unnecessary re-renders when there is no change to the state or context of your component. If the state and content of your component will ALWAYS change, React Memo becomes useless. Here are other points:

- Use React Memo if your component will render quite often.
- Use it when your component often renders with the same props. This happens to child components who are forced to re-render with the same props whenever the parent component renders.
- Use it in pure functional components alone. If you are using a class component, use the React.PureComponent.
- Use it if your component is big enough (contains a decent amount of UI elements) to have props equality check.

How to use React MemoAnchor

At this point, you now understand when to use React Memo and what it does. The major point is that you should use it when your component often renders with the same props.

Suppose you have a React application such as a Todo application in which you decide to break up the application into separate components to ensure it is easy to understand and use. You will have the parent component that holds your todo array in a state. The todo array will be passed as a prop to the Todo component:

```
// App.js
import React, { useState } from 'react';
import Todo from './Todo';

const App = () => {
   console.log('App component rendered');
   const [todo, setTodo] = useState([
      { id: 1, title: 'Read Book' },
      { id: 2, title: 'Fix Bug' },
   ]);
   const [text, setText] = useState('');

   const addTodo = () => {
      let newTodo = { id: 3, title: text };
      setTodo([...todo, newTodo]);
```

```
  };

  return (
    <div>
      <input
        type="text"
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <button type="button" onClick={addTodo}>
        Add todo
      </button>
      <Todo list={todo} />
    </div>
  );
};

export default App;
```

In the component above, there is a state to hold all the todo in an array, and then there is a form you would use to add a new todo to the array. Notice that at the beginning of the component, a console.log() statement will be implemented once your application renders.

The todo items are to be passed as props to the Todo component, which has been imported. This means that the Todo component is a child component of the App component. In the Todo component, the todo array that has been passed as a prop named list will be iterated so that you can access each item in the array and display:

```
// Todo.js
import React from 'react';
import TodoItem from './TodoItem';

const Todo = ({ list }) => {
  console.log('Todo component rendered');
  return (
    <ul>
      {list.map((item) => (
        <TodoItem key={item.id} item={item} />
      ))}
    </ul>
  );
};
export default Todo;
```

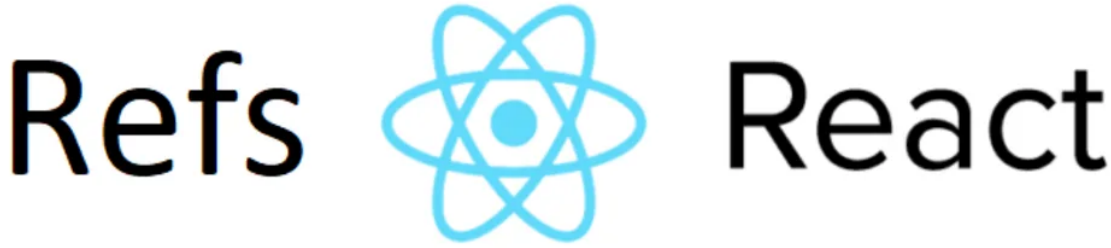
In the code above, a console.log() statement will display a text to show when the Todo component renders or re-renders. In the component above, each todo item is passed as a prop to a TodoItem component. This is done to properly explain what happens and when you need to memoize a component:

```
//TodoItem.js
const TodoItem = ({ item }) => {
   console.log('TodoItem component rendered');
   return <li>{item.title}</li>;
};
export default TodoItem;
```

The component above also has a console.log statement to help you know when it renders and re-renders. When you run your application and check the console, you will notice that all three components render.

Our component renders four times, once for the parent component (App.js), once for the Todo component, and finally, the TodoItem component renders twice because it has two elements. This means altogether, the application renders four times. This is normal for the initial render. But when you change something in the parent component that doesn't affect the child components, only the parent component should be rendered.

Introduction to Refs: Refs,



Refs is the shorthand used for references in React. It is similar to keys in React. It is an attribute which makes it possible to store a reference to particular DOM nodes or React elements. It provides a way to access React DOM nodes or React elements and how to interact with it. It is used when we want to change the value of a child component, without making the use of props.

When to Use Refs

Refs can be used in the following cases:

- When we need DOM measurements such as managing focus, text selection, or media playback.
- It is used in triggering imperative animations.
- When integrating with third-party DOM libraries.
- It can also be used in callbacks.

When to not use Refs

- Its use should be avoided for anything that can be done declaratively. For example, instead of using open() and close() methods on a Dialog component, you need to pass an isOpen prop to it.
- You should have to avoid overuse of the Refs.

How to create Refs

In React, Refs can be created by using React.createRef(). It can be assigned to React elements via the ref attribute. It is commonly assigned to an instance property when a component is created, and then can be referenced throughout the component.

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.callRef = React.createRef();
  }
  render() {
    return <div ref={this.callRef} />;
  }
```

```
}
```

How to access Refs

In React, when a ref is passed to an element inside the render method, a reference to the node can be accessed via the current attribute of the ref.

```
const node = this.callRef.current;
```

Refs current Properties

The ref value differs depending on the type of the node:

When the ref attribute is used in an HTML element, the ref created with React.createRef() receives the underlying DOM element as its current property.

If the ref attribute is used on a custom class component, then the ref object receives the mounted instance of the component as its current property.

The ref attribute cannot be used on function components because they don't have instances.

Add Ref to DOM elements

In the below example, we are adding a ref to store the reference to a DOM node or element.

```
import React, { Component } from 'react';
import { render } from 'react-dom';

class App extends React.Component {
   constructor(props) {
      super(props);
      this.callRef = React.createRef();
      this.addingRefInput = this.addingRefInput.bind(this);
   }

   addingRefInput() {
      this.callRef.current.focus();
   }

   render() {
      return (
         <div>
            <h2>Adding Ref to DOM element</h2>
            <input
               type="text"
               ref={this.callRef} />
            <input
               type="button"
               value="Add text input"
```

```
            onClick={this.addingRefInput}
          />
        </div>
      );
   }
}
export default App;
```

Add Ref to Class components

In the below example, we are adding a ref to store the reference to a class component.

Example

```
import React, { Component } from 'react';
import { render } from 'react-dom';

function CustomInput(props) {
   let callRefInput = React.createRef();

   function handleClick() {
      callRefInput.current.focus();
   }

   return (
      <div>
         <h2>Adding Ref to Class Component</h2>
         <input
            type="text"
            ref={callRefInput} />
         <input
            type="button"
            value="Focus input"
            onClick={handleClick}
         />
      </div>
   );
}
class App extends React.Component {
   constructor(props) {
      super(props);
      this.callRefInput = React.createRef();
   }

   focusRefInput() {
      this.callRefInput.current.focus();
   }
```

```
  render() {
    return (
      <CustomInput ref={this.callRefInput} />
    );
  }
}
export default App;
```

Callback refs

In react, there is another way to use refs that is called "callback refs" and it gives more control when the refs are set and unset. Instead of creating refs by createRef() method, React allows a way to create refs by passing a callback function to the ref attribute of a component. It looks like the code below.

```
<input type="text" ref={element => this.callRefInput = element} />
```

The callback function is used to store a reference to the DOM node in an instance property and can be accessed elsewhere. It can be accessed as below:

```
this.callRefInput.value
```

The example below helps to understand the working of callback refs.

```
import React, { Component } from 'react';
import { render } from 'react-dom';

class App extends React.Component {
  constructor(props) {
    super(props);

    this.callRefInput = null;

    this.setInputRef = element => {
      this.callRefInput = element;
    };

    this.focusRefInput = () => {
      //Focus the input using the raw DOM API
      if (this.callRefInput) this.callRefInput.focus();
    };
  }

  componentDidMount() {
```

```
    //autofocus of the input on mount
    this.focusRefInput();
  }

  render() {
    return (
      <div>
        <h2>Callback Refs Example</h2>
        <input
          type="text"
          ref={this.setInputRef}
        />
        <input
          type="button"
          value="Focus input text"
          onClick={this.focusRefInput}
        />
      </div>
    );
  }
}
export default App;
```

In the above example, React will call the "ref" callback to store the reference to the input DOM element when the component mounts, and when the component unmounts, call it with null. Refs are always up-to-date before the componentDidMount or componentDidUpdate fires. The callback refs passed between components is the same as you can work with object refs, which is created with React.createRef().

Forwarding Refs:

The forwardRef method in React allows parent components to move down (or “forward”) refs to their children. ForwardRef gives a child component a reference to a DOM entity created by its parent component in React. This helps the child to read and modify the element from any location where it is used.

How does forwardRef work in React?

In React, parent components typically use props to transfer data down to their children. Consider you make a child component with a new set of props to change its behavior. We need a way to change the behavior of a child component without having to look for the state or re-rendering the component. We can do this by using refs. We can access a DOM node that is represented by an element using refs. As a result, we will make changes to it without affecting its state or having to re-render it.

When a child component needs to refer to its parent’s current node, the parent component must have a way for the child to receive its ref. The technique is known as ref forwarding.

Syntax:

```
React.forwardRef((props, ref) => {})
```

Parameters: It takes a function with props and ref arguments.
Return Value: This function returns a JSX Element.

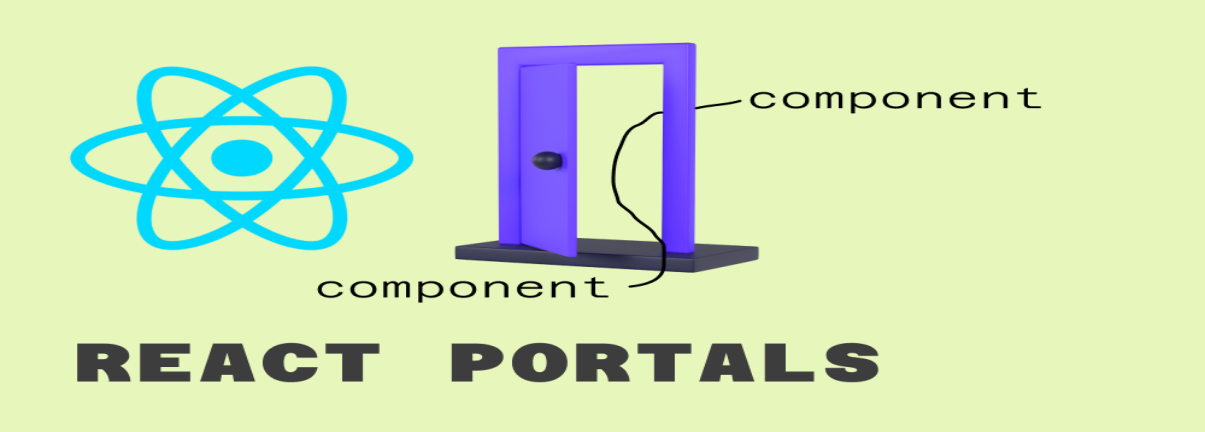
```
import React from 'react'

class App extends React.Component {
  constructor(props) {
    super(props)
    this.aRef = React.createRef()
  }
  render() {
    return (
      <>
        <Counter ref={this.aRef} />
        <button onClick={() => { console.log(this.aRef) }}>
          Ref
        </button>
      </>
    )
  }
}

const Counter = React.forwardRef((props, ref) => {
  class Counter extends React.Component {
    constructor(props) {
      super(props)
      this.state = {
        count: 0
      }
    }
    render() {
      return (
        <div>
          Count: {this.state.count}
          <button ref={ref} onClick={() => this.setState(
            { count: this.state.count + 1 })}>
            Increment
          </button>
        </div>
      )
    }
  }
```

```
    return <Counter />
})

export default App
```

Explanation: Since the counter is within the function, it can access the props and ref parameters using a closure. The counter is rendered and returned. The ref passed to the Counter component is set to the value of the button element's ref attribute. The counter element's ref attribute will now be set to refer to the button element.

The Counter component is rendered by the App component. It starts by creating a ref this.aRef is assigned to the Counter component's ref attribute as a value. We've included a button that logs the value of this.aRef. The this.aRef will hold the HTMLButtonElement of the Incr button in the Counter component. Clicking on the Ref button will confirm that It will log this.aRef which will log the object of the Incr button HTMLButtonElement. It didn't log the instance of the Counter because the Counter component forwarded it to its child component, the Increment button.

React portals:



React portals come up with a way to render children into a DOM node that occurs outside the DOM hierarchy of the parent component. The portals were introduced in the React 16.0 version.

So far we were having one DOM element in the HTML into which we were mounting our react application, i.e., the root element of our index.html in the public folder. Basically, we mount our App component onto our root element. It is almost a convention to have a div element with the id of root to be used as the root DOM element. If you take a look at the browser in the DOM tree every single React component in our application falls under the root element, i.e., inside this statement.

```
<div id="root"></div>
```

But React Portals provide us the ability to break out of this dom tree and render a component onto a dom node that is not under this root element. Doing so breaks the convention where a component needs to be rendered as a new element and follow a parent-child hierarchy. They are commonly used in modal dialog boxes, hovercards, loaders, and popup messages.

Syntax:

```
ReactDOM.createPortal(child, container)
```

Parameters: Here the first parameter is a child which can be a React element, string, or a fragment and the second parameter is a container which is the DOM node (or location) lying outside the DOM hierarchy of the parent component at which our portal is to be inserted.

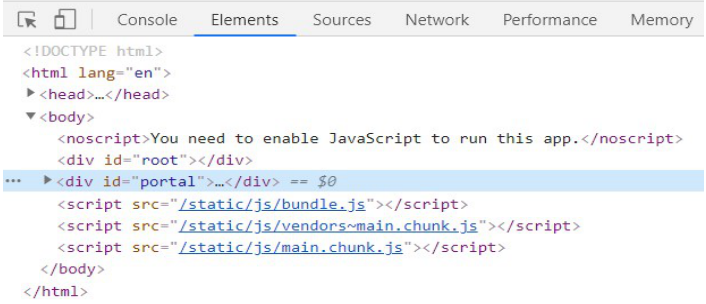
Importing: To create and use portals you need to import ReactDOM as shown below.

Example

```
//App.js
import ReactDOM from 'react-dom'
function App() {
  // Creating a portal
  return ReactDOM.createPortal(
```

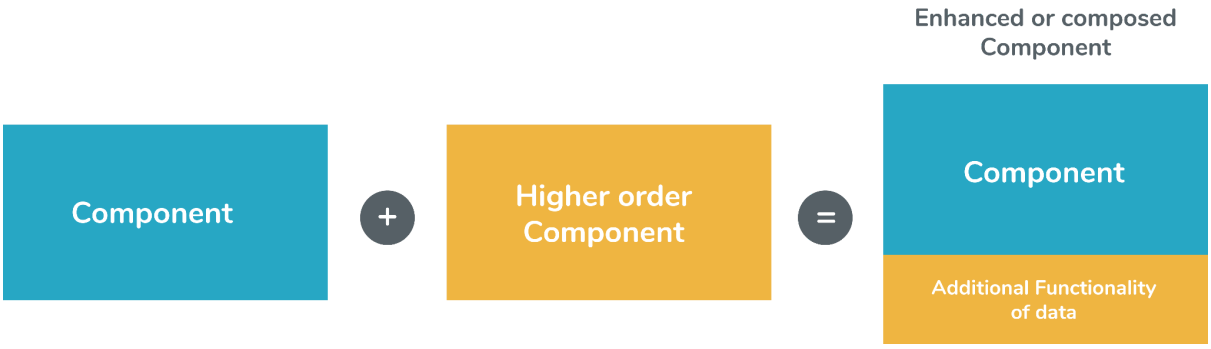
```
   <h1>Portal demo</h1>,
   document.getElementById('portal')
 )
}
export default App;
```

```
<!-- index.htm–>
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="utf-8" />
  <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta name="theme-color" content="#000000" />
  <meta
   name="description"
   content="Web site created using create-react-app"
  />
  <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
  <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
  <title>React App</title>
 </head>
 <body>
  <noscript>
   You need to enable JavaScript to run this app.
  </noscript>
  <div id="root"></div>
      <!-- new div added to access the child component -->
  <div id="portal"></div>
 </body>
</html>
```



Explanation: Here, we can see that our <h1> tag Portal Demo is under the newly created portal DOM node, not the traditional root DOM node. It clearly tells us how React Portal provides the ability to break out of the root DOM tree and renders a component/element outside the parent DOM element.

Components: Higher Order Components



It is also known as HOC. In React, HOC is an advanced technique for reusing component logic. It is a function that takes a component and returns a new component. According to the official website, it is not the feature(part) in React API, but a pattern that emerges from React compositional nature. They are similar to JavaScript functions used for adding additional functionalities to the existing component.

A higher order component function accepts another function as an argument. The map function is the best example to understand this. The main goal of this is to decompose the component logic into simpler and smaller functions that can be reused as you need.

Syntax:

```
const NewComponent = higherOrderComponent(WrappedComponent);
```

We know that a component transforms props into UI, and a higher-order component converts a component to another component and allows to add additional data or functionality into this. Hocs are common in third-party libraries. The examples of HOCs are Redux's connect and Relay's createFragmentContainer.

Now, we can understand the working of HOCs from the below example.

```
//Function Creation
function add(a, b) {
   return a + b
}
function higherOrder(a, addReference) {
   return addReference(a, 20)
}
//Function call
higherOrder(30, add) // 50
```

In the above example, we have created two functions add() and higherOrder(). Now, we provide the add() function as an argument to the higherOrder() function. For invoking, rename it addReference in the higherOrder() function, and then invoke it.

Here, the function you are passing is called a callback function, and the function where you are passing the callback function is called a higher-order(HOCs) function.

Example

Create a new file with the name HOC.js. In this file, we have made one function HOC. It accepts one argument as a component. Here, that component is App.

```
//HOC.js
import React, { Component } from 'react';
export default function Hoc(HocComponent) {
   return class extends Component {
      render() {
         return (
            <div>
               <HocComponent></HocComponent>
            </div>

         );
      }
   }
}
```

Now, include the HOC.js file into the App.js file. In this file, we need to call the HOC function.

```
App = Hoc(App);
```

The App component is wrapped inside another React component so that we can modify it. Thus, it becomes the primary application of the Higher-Order Components.

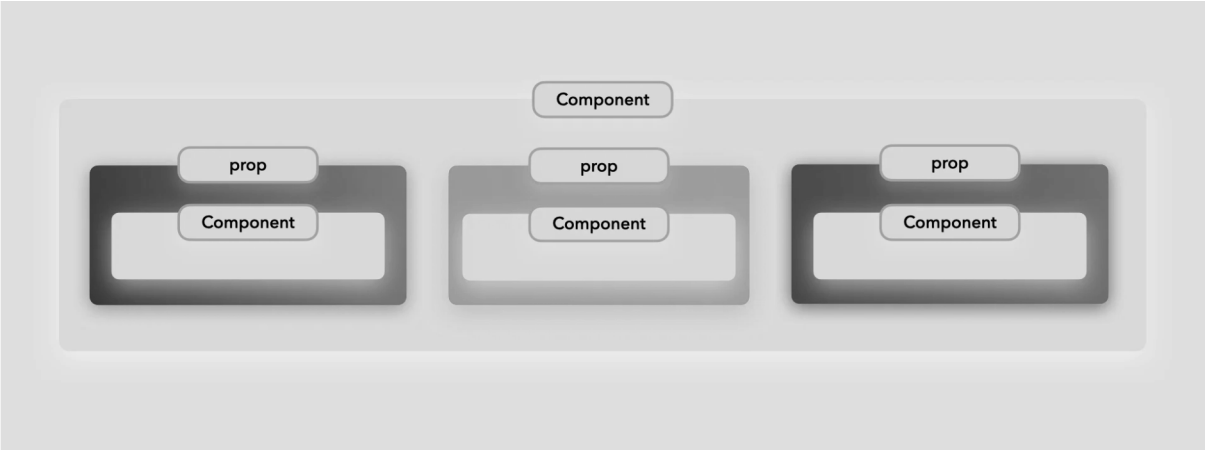
```
//App.js
import React, { Component } from 'react';
import Hoc from './HOC';

class App extends Component {
  render() {
    return (
      <div>
        <h2>HOC Example</h2>
        Geetanjali provides best CS tutorials.
      </div>
    )
  }
}
App = Hoc(App);
export default App;
```

Higher-Order Component Conventions
Do not use HOCs inside the render method of a component.

The static methods must be copied over to have access to them. You can do this using the hoist-non-react-statics package to automatically copy all non-React static methods.

HOCs do not work for refs as 'Refs' does not pass through as a parameter or argument. If you add a ref to an element in the HOC component, the ref refers to an instance of the outermost container component, not the wrapped component.

Props Again!: Rendering Props and Context



Render Props in React is something where a component's prop is assigned a function and that is called in the render method of the component. Calling the function will return a React element or component. Third-party libraries like React Router and Downshift use this approach.

A render prop is a function prop that the component uses to know what to render. Through this, we can also pass a state to another.

Example

Create a Movement.js file and code as below,

```
import React, { Component } from 'react'

export class Movement extends Component {

  constructor(props) {
    super(props)

    this.state = {
      x: 0,
      y: 0
    }
  }

  onMouseMove = (event) => {
    this.setState({
      x: event.clientX,
      y: event.clientY
    })
  }

  render() {
    const { x, y } = this.state
    return (
```

```
        <div style={{ height: '100%' }} onMouseMove={this.onMouseMove}>
          <h1> The current mouse position is ({x},{y})</h1>
        </div>
      )
    }
}

export default Movement
```

Now add this Movement.js in App.js,

```
import React from 'react';
import './App.css';
import './css/custom.css';
import Movement from './components/Movement'

function App() {
  return (
    <div>
      <Movement></Movement>
    </div>
  );
}

export default App;
```

Now let's go further with the demo, we will create a component named Monkey in which on moving the mouse, the monkey will also move following the banana.

Change the code of Movement.js as below.

```
import React, { Component } from 'react'
import PropTypes from 'prop-types'

export class Movement extends Component {

  static propTypes = {
    render: PropTypes.func.isRequired
  }

  state = { x: 0, y: 0 }

  onMouseMove = (event) => {
    this.setState({
      x: event.clientX,
      y: event.clientY
```

```
      })
   }

   render() {
      return (
         <div onMouseMove={this.onMouseMove}>
            {this.props.render(this.state)}
         </div>
      )
   }
}
export default Movement
```

Now, create Monkey.js with the below code. Below, we also need 2 images of monkey and banana respectively which are uploaded in the source code of Demo2.

```
import React, { Component } from 'react'
import monkey from '../images/monkey.jpg'
import banana from '../images/banana.jpg'

export class Monkey extends Component {
   render() {
      const { x, y } = this.props.Movement

      return (
         <div>
            <img src={monkey} alt='monkey' style={{ position: 'relative', left: x, right: y,
height: 150 }}></img>
            <img src={banana} alt='banana' style={{ position: 'relative', left: (x + 10), right: (y
+ 10), height: 30 }}></img>
         </div>
      )
   }
}

export default Monkey
```

Now, in the final step, call Monkey.js in App.js.

```
import React from 'react';
import './App.css';
import './css/custom.css';
import Movement from './components/Movement'
import Monkey from './components/Monkey'

function App() {
```

```
  return (
    <div>
      <Movement render={({ x, y }) => (
        <Monkey Movement={{ x, y }} />
      )} />

    </div>
  );
}

export default App;
```

In the above program, on moving the mouse, the monkey will also move. The above demo is quite interesting and lets us allow other animation to be created using react-motion API.

Context

Context in React.js is the concept of passing data through a component tree without passing props down manually to each level.

In basic React applications, to pass data from parent to child is done using props but this is a heavy approach if data needs to be passed on multiple levels like passing username or theme required by most of the components in the application. Context lets you provide the functionality to share values among components without explicitly passing props through every level of the component tree.

Mainly context is used when the current authentication, themes, or preferred language need to be passed to the child levels when not required to pass props.

To get data using context we need to follow 3 steps,

- We need to first create the context.
- Then we need to provide value to the context.
- Lastly, we need to consume the value of context.

Now let's have a look at the demo,

For Example - In the current demo, we will see how to use username in child components using context.

First, we will create a new file namely UserContext.js. This is the first step where we are

```
import React from 'react'

const UserContext = React.createContext()

const UserProvider = UserContext.Provider
const UserConsumer = UserContext.Consumer

export {UserProvider,UserConsumer}
```

Now, we will create a nested structure in which I will create 3 level components Master Component -> Home Component -> Profile Component.

App.js in which we will execute second step we need to provide value to consumer,

```
import React from 'react';
import './App.css';
import './css/custom.css';
import MasterComponent from './components/MasterComponent'
import { UserProvider } from './components/UserContext';

function App() {
  return (
```

```
      <div className="App">
        <UserProvider value="New User">
          <MasterComponent />
        </UserProvider>
      </div>
    );
}

export default App;
```

MasterComponent.js

```
import React, { Component } from 'react'
import HomeComponent from './HomeComponent'
class MasterComponent extends Component {
   render() {
      return (
        <div>
          <HomeComponent />
        </div>
      )
   }
}

export default MasterComponent
```

Now second level component is named HomeComponent.js

```
import React, { Component } from 'react'
import ProfileComponent from './ProfileComponent'

class HomeComponent extends Component {
   render() {
      return (
        <div>
          <ProfileComponent/>
        </div>
      )
   }
}
export default HomeComponent
```

And third level Component named ProfileComponent in which we are going to consume context value

```
import React, { Component } from 'react'
```

```
import { UserConsumer } from './UserContext';

class ProfileComponent extends Component {
   render() {
      return (
         <UserConsumer>
            {
               (username) => {
                  return <div>Hello {username}</div>
               }
            }
         </UserConsumer>
      )
   }
}
export default ProfileComponent
```

If in UserContext.js while creating context we will define default value and if in case we don't provide any value in component then it will take default value from UserContext,

```
//And App.js code will be as below,
import React from 'react';
import './App.css';
import './css/custom.css';
import MasterComponent from './components/MasterComponent'
import { UserProvider } from './components/UserContext';

function App() {
   return (
      <div className="App">
         {/* <UserProvider> */}
         <MasterComponent />
         {/* </UserProvider> */}
      </div>
   );
}
export default App;
```

While using context we can only pass one value, for multiple values we need to use multiple context.
There is one more way to access context value using Context Type property.

First we need to export UserContext from UserContext.js file,

```
import React from 'react'
```

```
const UserContext = React.createContext('Guest')
const UserProvider = UserContext.Provider
const UserConsumer = UserContext.Consumer

export {UserProvider,UserConsumer}
export default UserContext
```

Now we will consume context value using contextType in HomeComponent.js

```
import React, { Component } from 'react'
import ProfileComponent from './ProfileComponent'
import UserContext from './UserContext'

class HomeComponent extends Component {
   render() {
      return (
         <div>
            Home Component {this.contextType}
            <ProfileComponent/>
         </div>
      )
   }
}

HomeComponent.contextType = UserContext

export default HomeComponent
```


Accessing multiple context values,

```
import React from 'react'
const UserContext = React.createContext('Guest')
const ThemeContext = React.createContext('theme')
const UserProvider = UserContext.Provider
const UserConsumer = UserContext.Consumer
const ThemeProvider = ThemeContext.Provider
const ThemeConsumer = ThemeContext.Consumer
export {UserProvider,UserConsumer,ThemeConsumer,ThemeProvider}

export default UserContext
```

and using it in App.js,

```
import React from 'react';
import './App.css';
import './css/custom.css';
```

```
import MasterComponent from './components/MasterComponent'
import { UserProvider, ThemeProvider } from './components/UserContext';

function App() {
   return (
      <div className="App">
         <UserProvider value="New User">
            <ThemeProvider value="red">
               <MasterComponent />
            </ThemeProvider>
         </UserProvider>
      </div>
   );
}

export default App;
```

Now consuming value of ThemeProvider in ProfileComponent.js file,

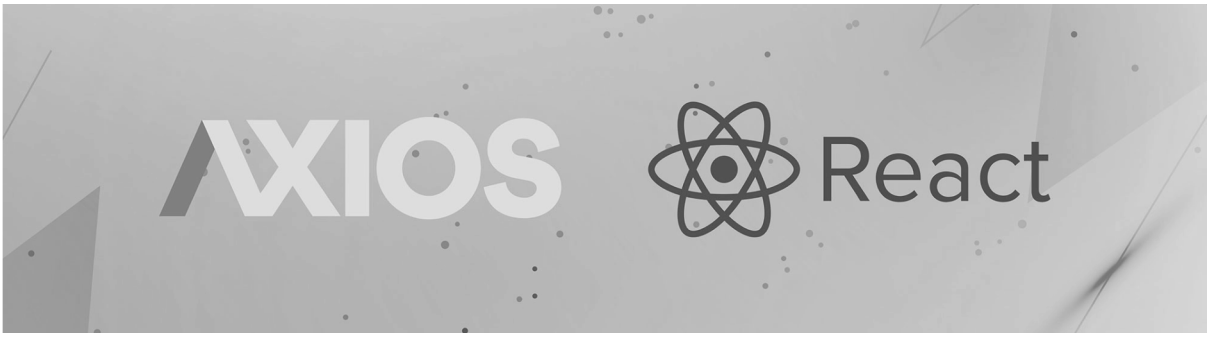
```
import React, { Component } from 'react'
import { UserConsumer,ThemeConsumer } from './UserContext';

class ProfileComponent extends Component {
   render() {
      return (
        <UserConsumer>
          {
             username => (
              <ThemeConsumer>
              {color => (
                  <div style={{ color: color }}>Hello {username}</div>
              )}
              </ThemeConsumer>
          )}
        </UserConsumer>
      )
   }
}
export default ProfileComponent
```

Limitation of Context

Context can only be used with the class component.

Using context type only a single context value can be accessed. For accessing multiple values, we will need to use nested consumer context.

HTTP



Basics of HTTP and React

So far, we have learned about the basics of React and how props and state are used. Now, we will learn how React makes API calls to fetch data and display in the browser. As React is just a library for the user interface, it only knows about Props and State so here, HTTP comes into the picture.

To fetch data in React, there are many HTTP libraries available like - **Apollo**, **Axios**, **Relay Modern**, **Request**, and **Superagent**. In this article, we are going to use Axios library.

To install axios

```
npm install axios
```

After the command is executed successfully, we will see in the package.json file that the dependencies are updated.

Now, we will see how we can fetch data and display it in the browser.

HTTP GET

Here, we are using an online API to fetch data from it instead of creating our own API. We are using **https://reqres.in/** to test our application.

Create a component named DisplayList.js.

```
import React, { Component } from 'react'
export class DisplayList extends Component {
  render() {
    return (
      <div>
        List of Users
</div>
    )
  }
}
export default DisplayList
```

And include this component in App.js.

```
import React from 'react';
import logo from './logo.svg';
import './App.css';
import DisplayList from './components/DisplayList'
function App() {
  return (
    <div className="App">
      <DisplayList></DisplayList>
    </div>
  );
}
export default App;
```

Now, we will fetch data from the library using Axios. For that, first, we need to import the axios library.

```
import axios from 'axios'
Now, to store the data that we get from API, we will create an empty array in state.
this.state = {
    Users:[]
}
```

Now, we will call the API and store the data in our array in componentDidMount() lifecycle method. This is executed when the component mounts for the first time and it is called only once.

So, the component will go like below.

```
import React, { Component } from 'react'
import axios from 'axios'

const error ={
   color: 'red',
   fontWeight:'bold',
   fontSize:'14'
}

export class DisplayList extends Component {
   constructor(props) {
      super(props)

      this.state = {
         users:[],
          errorMessage:""
```

```
    }
  }

  componentDidMount(){
    axios.get('https://reqres.in/api/users/')
    .then(response => {
      console.log(response.data.data)
      this.setState({
        users:response.data.data
      })
    })
    .catch(error =>{
      this.setState({
        errorMessage:"Error fetching data"
      })
    })
  }
  render() {
    const {users,errorMessage} = this.state
    console.log(users.length)
    return (
      <div>
        List of Users
        {
          users.length?
          users.map(user=> <div key={user.id}><img src={user.avatar}
alt={user.first_name}/>{user.first_name + " "+ user.last_name}</div>):
          null
        }
        {errorMessage ? <div style={error}>{errorMessage}</div> : null}
      </div>
    )
  }
}

export default DisplayList
```

In the above, componentDidMount() will be called once the component is mounted, so it will display a list of users from the provided API. If the API is incorrect or does not return output, then the error message will be displayed.

HTTP and React

HTTP POST

Like HTTP Get, we use the Axios Post method to send the data to the API we use.

Example:

```
import React, { Component } from 'react'
import axios from 'axios'

class UserForm extends Component {
  constructor(props) {
    super(props)

    this.state = {
      first_name: "",
      last_name: "",
      email: ""
    }
  }

  onChangeHandler = (e) => {
    this.setState({ [e.target.name]: e.target.value })
  }

  onSubmitHandler = (e) => {
    e.preventDefault();
    console.log(this.state)
    axios.post('https://reqres.in/api/users/',this.state)
    .then(response=>{
      console.log(response)
    })
    .catch(error => {
      console.log(error)
    })
  }

  render() {
    const { first_name, last_name, email } = this.state
    return (
      <div>
        <form onSubmit={this.onSubmitHandler}>
          <div>
            Email : <input type="text" name="email" value={email}
onChange={this.onChangeHandler} />
          </div>
          <div>
            First Name : <input type="text" name="first_name" value={first_name}
onChange={this.onChangeHandler} />
```

```
            </div>
            <div>
              Last Name : <input type="text" name="last_name" value={last_name}
onChange={this.onChangeHandler} />
            </div>
            <button type="submit">Submit</button>
          </form>
        </div>
      )
   }
}

export default UserForm
```

Let us include the UserForm component in App.js.

```
import React from 'react';
import './App.css';
import UserForm from './components/UserForm'

function App() {
  return (
    <div className="App">
      <UserForm></UserForm>
    </div>
  );
}

export default App;
```


We can also use the PUT, DELETE function in the same way.

Introduction: React Hooks introduction

If you don't like classes then hooks are here to help you. Hooks are functions that enable you to use React functionalities like state and lifecycle features without using classes. As the name suggests, it enables you to hook into React state and lifecycle features from function components.

Hooks do not work inside classes and are backward-compatible, meaning they have no breaking changes. So, it is your choice if you want to use them or not. This new feature gives you the power to use all React features, even the function components.

Make sure you write hooks in standard follow form. Call hooks at the top level of React functions to ensure that hooks are called in the same order every time. Try to avoid calling them in loops, nested functions, or inside conditions. Also, make sure you call them from the React function component, not from JavaScript functions. If you don't follow that rule then it may result in some unexceptional behaviors. React also provides a linter plugin to ensure that these rules apply automatically.

You don't have to install anything to use hooks. They come with React from the 16.8 version onward.

useState Hook

To declare, change and use states using hooks we have useState() function component. It returns two value pairs, the current state, and a function to update the state. It is something like this.setState in a class but it is different from this.setState by the fact that it does not combine the old and new state. UseState() can be called inside a function component or from an event handler. The initial state is passed inside useState() as an argument which is used during the first render. There is no bound on using multiple state hooks in a single component; we can use as many as we want.

```
import React, { useState } from 'react';

export default function App() {
  const [counter, setCounter] = useState(1);
  const incrBy3 = () => {
    setCounter(counter + 3);
  };
  return (
    <div className="container">
      <h1>Increment By 3</h1>
      <div className="counter-box">
        <span>{counter}</span>
        <button onClick={incrBy3}>+ 3</button>
      </div>
    </div>
  );
}
```

Interesting Facts of useState Hook

A few points to emphasize here that we often ignore.

With the useState hook, the state gets created only at the first render using the initial value we pass as an argument to it.
For every re-render (subsequent renders after the initial render), ReactJS ignores the initial value we pass as the argument. In this case, it returns the current value of the state.
ReactJS provides us with a mechanism to get the previous state value when dealing with the current state value.
That's all about the interesting facts, but they may not make much sense without understanding their advantages. So, there are two primary advantages,

We can perform a lazy initialization of the state.
We can use the previous state value alongside the current one to solve a use case.

How to perform Lazy Initialization of the State?

If the initial state value is simple data like a number, string, etc., we are good with how we have created and initialized the state in the above example. At times, you may want to initialize the state with a computed value. The computation can be an intense and time-taking activity.

With the useState hook, you can pass a function as an argument to initialize the state lazily. As discussed, the initial value is needed only once at the first render. There is no point in performing this heavy computation on the subsequent renders.

```
const [counter, setCounter] = useState(() => Math.floor(Math.random() * 16));
```

The code snippet above lazily initializes the counter state with a random number. Please note, you don't have to do this always, but the knowledge is worthy. Now you know you have a way to perform lazy state initialization.

useState Previous state

The useState hook returns a function to update the state. In our example, we know it as the setCounter(value) method. A specialty of this method is, you can get the previous(or old) state value to update the state. Please take a look into the code snippet below,

```
const incrBy3 = () => {
   setCounter((prev) => prev + 3);
};
```

Here we pass a callback function to the setCounter() method that gives us the previous value to use. Isn't that amazing?

useState with object

One of React's most commonly used Hooks is useState, which manages states in React projects as well as objects' states. With an object, however, we can't update it directly or the component won't rerender.

To solve this problem, we'll look at how to use useState when working with objects, including the method of creating a temporary object with one property and using object destructuring to create a new object from the two existing objects.

In the following code sample, we'll create a state object, shopCart, and its setter, setShopCart. shopCart then carries the object's current state while setShopCart updates the state value of shopCart

```
const [shopCart, setShopCart] = useState({});

let updatedValue = {};
updatedValue = { "item1": "juice" };
setShopCart(shopCart => ({
  ...shopCart,
  ...updatedValue
}));
```

We can then create another object, updatedValue, which carries the state value to update shopCart.

By setting the updatedValue object to the new {"item1":"juice"} value, setShopCart can update the value of the shopCart state object to the value in updatedValue.

useState with array

Arrays are mutable in JavaScript, but you should treat them as immutable when you store them in state. Just like with objects, when you want to update an array stored in state, you need to create a new one (or make a copy of an existing one), and then set state to use the new array.'

In React, Let's first create a friends array. We will have two properties, name, and age.

```
const friendsArray = [
   {
      name: "John",
      age: 19,
   },
   {
      name: "Candy",
      age: 18,
   },
   {
      name: "mandy",
      age: 20,
   },
];
```

Now let's work with this array and useState

```
import { useState } from "react";

const App = () => {
```

```
  const [friends, setFriends] = useState(friendsArray); // Setting default value

  const handleAddFriend = () => {
    ...
  };

  return (
    <main>
      <ul>
        // Mapping over array of friends
        {friends.map((friend, index) => (
          // Setting "index" as key because name and age can be repeated, It will be
better if you assign uniqe id as key
          <li key={index}>
            <span>name: {friend.name}</span>{" "}
            <span>age: {friend.age}</span>
          </li>
        ))}
        <button onClick={handleAddFriend}>Add Friends</button>
      </ul>
    </main>
  );
};
export default App;
```

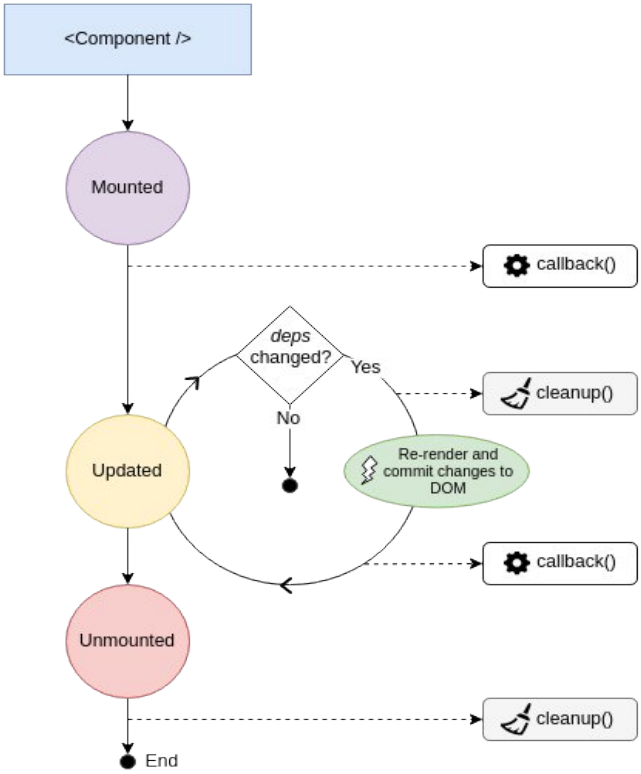
Here, we are mapping over a friends array and displaying it.

Let's now see How to add new values to this array

```
const handleAddFriend = () => {
  setFriends((prevFriends) => [
    ...prevFriends,
    {
      name: "Random Friend Name",
      age: 20, // Random age
    },
  ]);
};
```

Here, setState lets us define an anonymous function that has its previews state as an argument to the function, then we are using spread operator to get our all previews value(state) now after this we can add our new value.

useEffect: useEffect Hook



To perform side effects operations like changing DOM, data fetching, etc., from React function components we use Effect hook, i.e., **useEffect**. These operations are called so since they can affect other components and can't be performed during rendering. **UseEffect** hook provides us the power of **componentDidMount**, **componentDidUpdate**, and **componentWillUnmount**.

UseEffect is declared inside the components because they need to have access to their state and props. UseEffect can be run at first render, after every render, or used to specify clean up based on how we declare them. By default, they run after every render including the first render.

```
import React, {
  useState,
  useEffect
} from 'react';

function usingEffect() {
  const [count, setCount] = useState(0);

  /* default behavior is similar to componentDidMount and componentDidUpdate: */
  useEffect(() => {
```

```
  // Change document title
  document.title = `Clicked {count} times`;
 });

 return (
  <div>
   <p>Clicked count = {count}</p>
   <button onClick={() => setCount(count + 1)}>
    Button
   </button>
  </div>
 );
}
```

In the above example we are updating the document title every time the count gets updated. Here useEffect is working similarly to componentDidMount and componentDidUpdate combined since it will run during the first render and also after every update. This is the default behavior of useEffect, but it can be changed. Let's see how.

useEffect after render

If you want to use useEffect as componentDidMount then you can pass an empty array [] in the second argument to useEffect as in the below example:

```
import React, {
 useState,
 useEffect
} from 'react';

function usingEffect() {
 const [count, setCount] = useState(0);

 // Similar to componentDidMount
 useEffect(() => {
  // Only update when component mount
  document.title = `You clicked {count} times`;
 }, []);

 return (
  <div>
   <p>Your click count is {count}</p>
   <button onClick={() => setCount(count + 1)}>
    Button
```

```
    </button>
   </div>
  );
}
```

Conditionally run effects, run effects only once

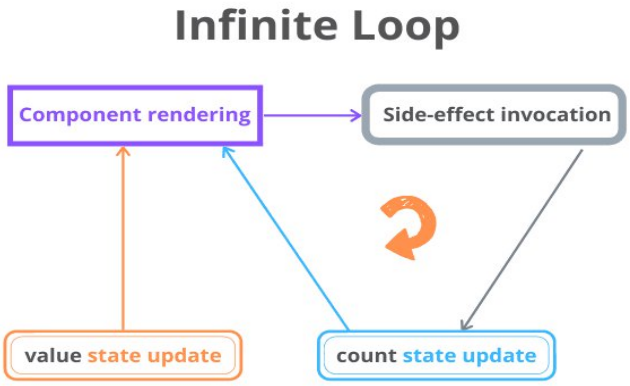
If you want to use it as componentDidUpdate, which only re-renders if a specific state is updated, then we can pass that state value in the second argument of useEffect as below:

```
useEffect(() => {
  document.title = `Clicked {count} times`;
}, [count]); /* if count changes then only effect is re-run */
The above code is similar to the below code when we use classes.

componentDidUpdate(prevProps, prevState) {
  if (prevState.count !== this.state.count) {
    document.title = `Clicked {this.state.count} times`;
  }
}
```

useEffect with cleanup,
If you also want it to perform the componentWillUnmount function, (which can be used for clean up), then we can return value from useEffect.

```
useEffect(() => {
  function handleStatusChange(status) {
    setStatus(status);
  }

  API.changeStatusToTrue(props.status, handleStatusChange);
  return () => {
    /* for implementing componentDidUnmount behaviour */
    API.changeStatusToFalse(props.status, handleStatusChange);
  };
}, [props.status]); /* re-run only if props.status changes */
```

useEffect with incorrect dependency



useEffect() hook manages the side-effects like fetching over the network, manipulating DOM directly, and starting/ending timers.

Although the useEffect() is one of the most used hooks along with useState(), it requires time to familiarize and use correctly.

A pitfall you might experience when working with useEffect() is the infinite loop of component renderings. In this post, I'll describe the common scenarios that generate infinite loops and how to avoid them.

Let's say you want to create a component having an input field, and also display how many times the user changed that input.

```
import { useEffect, useState } from 'react';

function CountInputChanges() {
  const [value, setValue] = useState('');
  const [count, setCount] = useState(-1);

  useEffect(() => setCount(count + 1));
  const onChange = ({ target }) => setValue(target.value);
  return (
    <div>
      <input type="text" value={value} onChange={onChange} />
      <div>Number of changes: {count}</div>
    </div>
  )
}
```

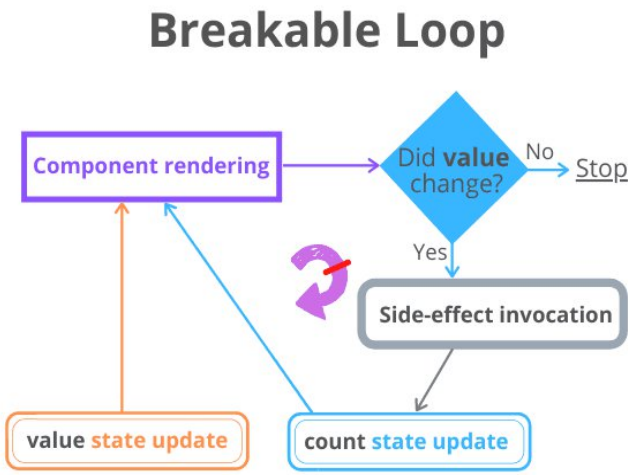
After initial rendering, useEffect() executes the side-effect callback and updates the state. The state update triggers re-rendering. After re-rendering useEffect() executes the side-effect callback and again updates the state, which triggers again a re-rendering. ...and so on indefinitely.

The infinite loop is fixed with correct management of the useEffect(callback, dependencies) dependencies argument.

Because you want count to increment when value changes, you can simply add value as a dependency of the side-effect:

```
import { useEffect, useState } from 'react';
function CountInputChanges() {
  const [value, setValue] = useState('');
  const [count, setCount] = useState(-1);
  useEffect(() => setCount(count + 1), [value]); // Solution
  const onChange = ({ target }) => setValue(target.value);
  return (
    <div>
      <input type="text" value={value} onChange={onChange} />
      <div>Number of changes: {count}</div>
    </div>
  );
}
```

By adding [value] as a dependency of useEffect(..., [value]), the count state variable will only be updated when [value] changes. This solves the infinite loop.



Now, as soon as you type into the input field, the count state correctly displays the number of input value changes.

Fetching data:
Fetching data with useEffect

Side-effects then, are operations that change things outside of your function, making the function impure.

Fetching data from an API, communicating with a database, and sending logs to a logging service are all considered side-effects, as it's possible to have a different output for the same input. For example, your request might fail, your database might be unreachable, or your logging service might have reached its quota.

This is why useEffect is the hook for us - by fetching data, we're making our React component impure, and useEffect provides us a safe place to write impure code.

You might notice a few things are missing from this example:

- we're not doing anything with the data once we fetch it
- we've hardcoded the URL to fetch data from

```
useEffect(() => {
  const fetchData = async () => {
    const response = await fetch(`https://swapi.dev/api/people/1/`);
    const newData = await response.json();
  };

  fetchData();
});
```

To make this useEffect useful, we'll need to:

- update our useEffect to pass a prop called id to the URL,
- use a dependency array, so that we only run this useEffect when id changes, and then
- use the useState hook to store our data so we can display it later

```
import React, { useEffect, useState } from 'react';

export default function DataDisplayer(props) {
  const [data, setData] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      const response = await fetch(`https://swapi.dev/api/people/${props.id}/`);
      const newData = await response.json();
```

```
    setData(newData);
   };

   fetchData();
  }, [props.id]);

  if (data) {
   return <div>{data.name}</div>;
  } else {
   return null;
  }
}
```

You might find fetching data in this way results in quite a bit of repeated code, especially if you're using fetch, and handle error and loading states. To avoid this, a common solution is to write a custom hook.

useContext Hook

React Context is a way to manage state globally.
It can be used together with the useState Hook to share state between deeply nested components more easily than with useState alone.

Create Context

To create context, you must Import createContext and initialize it:

```
import { useState, createContext } from "react";
import ReactDOM from "react-dom/client";

const UserContext = createContext()
```

Next we'll use the Context Provider to wrap the tree of components that need the state Context.

Context Provider

Wrap child components in the Context Provider and supply the state value.

```
function Component1() {
  const [user, setUser] = useState("Jesse Hall");

  return (
    <UserContext.Provider value={user}>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 user={user} />
```

```
    </UserContext.Provider>
  );
}
```

Use the useContext Hook

In order to use the Context in a child component, we need to access it using the useContext Hook.

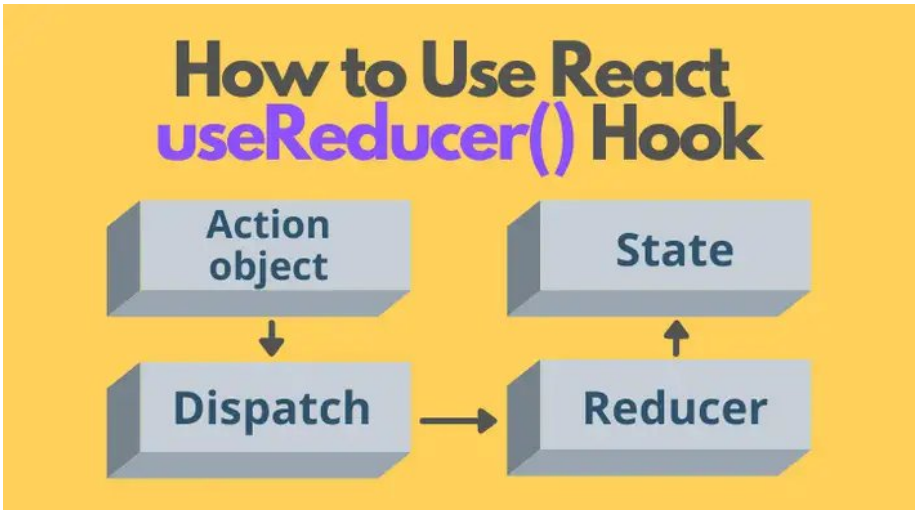
First, include the useContext in the import statement:

```
import { useState, createContext, useContext } from "react";
```

Then you can access the user Context in all components:

```
function Component5() {
  const user = useContext(UserContext);

  return (
    <>
      <h1>Component 5</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}
```

useReducer Hook:
useReducer – simple state and action



If you've used the useState() hook to manage a non-trivial state like a list of items, where you need to add, update and remove items in the state, you can notice that the state management logic takes a good part of the component's body.

A React component should usually contain the logic that calculates the output. But the state management logic is a different concern that should be managed in a separate place. Otherwise, you get a mix of state management and rendering logic in one place, and that's difficult to read, maintain, and test!

To help you separate the concerns (rendering and state management) React provides the hook useReducer(). The hook does so by extracting the state management out of the component.

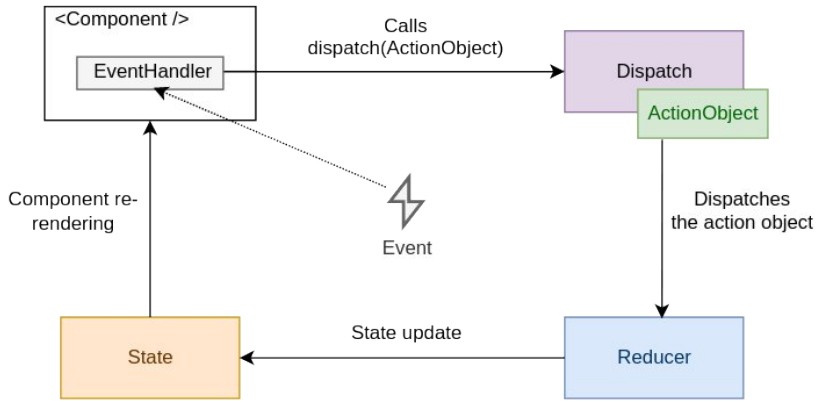
Let's see how the useReducer() hook works with accessible real-world examples.

```
import { useReducer } from 'react';
function MyComponent() {
  const [state, dispatch] = useReducer(reducer, initialState);
  const action = {
    type: 'ActionType'
  };
  return (
    <button onClick={() => dispatch(action)}>
      Click me
    </button>
  );
}
```

The following reducer function supports the increase and decrease of a counter state:

```
function reducer(state, action) {
  let newState;
  switch (action.type) {
    case 'increase':
      newState = { counter: state.counter + 1 };
      break;
    case 'decrease':
      newState = { counter: state.counter - 1 };
      break;
    default:
      throw new Error();
  }
  return newState;
}
```

complex state and action



The dispatch function with the action object is called as a result of an event handler or completing a fetch request.

Then React redirects the action object and the current state value to the reducer function.

The reducer function uses the action object and performs a state update, returning the new state.

React then checks whether the new state differs from the previous one. If the state has been updated, React re-renders the component, and useReducer() returns the new state value: [newState, ...] = useReducer(...).

multiple useReducers

We will use the same reducer function for multiple counters. We don't need to initialize the different states for the second counter. All useReducer hooks will behave as individual states.

```
import React, { useReducer } from "react";

const initialState = 0;

const reducer = (state, action) => {
  switch (action) {
    case 'increment':
      return state + 1;
    case 'decrement':
      return state - 1;
    case 'reset':
      return initialState;
    default:
      return state;
  }
}

function App() {
  const [count, dispatch] = useReducer(reducer, initialState);
  const [count2, dispatch2] = useReducer(reducer, initialState);

  return (
    <div classname="App">
      <h3>Multiple useReducer hook - <a href="https://www.cluemediator.com"
target="_blank" rel="noopener noreferrer">Clue Mediator</a></h3>
      <span>First Counter: {count}</span>
      <button onclick={() ==> dispatch('increment')}>Increment</button>
```

```
    <button onclick={() ==> dispatch('decrement')}>Decrement</button>
    <button onclick={() ==> dispatch('reset')}>Reset</button>
    <br /><br />
    <span>Second Counter: {count2}</span>
    <button onclick={() ==> dispatch2('increment')}>Increment</button>
    <button onclick={() ==> dispatch2('decrement')}>Decrement</button>
    <button onclick={() ==> dispatch2('reset')}>Reset</button>
   </div>
  );
}

export default App;

import React, { useReducer } from "react";

const initialState = 0;

const reducer = (state, action) => {
  switch (action) {
    case 'increment':
      return state + 1;
    case 'decrement':
      return state - 1;
    case 'reset':
      return initialState;
    default:
      return state;
  }
}

function App() {
  const [count, dispatch] = useReducer(reducer, initialState);
  const [count2, dispatch2] = useReducer(reducer, initialState);

  return (
    <div classname="App">
      <h3>Multiple useReducer hook - <a href="https://www.cluemediator.com"
target="_blank" rel="noopener noreferrer">Clue Mediator</a></h3>
      <span>First Counter: {count}</span>
      <button onclick={() ==> dispatch('increment')}>Increment</button>
      <button onclick={() ==> dispatch('decrement')}>Decrement</button>
      <button onclick={() ==> dispatch('reset')}>Reset</button>
      <br /><br />
      <span>Second Counter: {count2}</span>
```

```
    <button onclick={() ==> dispatch2('increment')}>Increment</button>
    <button onclick={() ==> dispatch2('decrement')}>Decrement</button>
    <button onclick={() ==> dispatch2('reset')}>Reset</button>
   </div>
  );
}

export default App;
```

useContext

useContext is a React Hook that lets you read and subscribe to context from your component.

```
useContext(SomeContext)
```

Call **useContext** at the top level of your component to read and subscribe to context.

```
import { useContext } from 'react';

function MyComponent() {
  const theme = useContext(ThemeContext);
  // ...
```

Parameters

SomeContext: The context that you've previously created with createContext. The context itself does not hold the information, it only represents the kind of information you can provide or read from components.

Returns

useContext returns the context value for the calling component. It is determined as the value passed to the closest SomeContext.Provider above the calling component in the tree. If there is no such provider, then the returned value will be the defaultValue you have passed to createContext for that context.
The returned value is always up-to-date. React automatically re-renders components that read some context if it changes.

Caveats

useContext() call in a component is not affected by providers returned from the same component. The corresponding <Context.Provider> needs to be above the component doing the useContext() call.

React automatically re-renders all the children that use a particular context starting from the provider that receives a different value. The previous and the next values are compared with the Object.is comparison. Skipping re-renders with memo does not prevent the children receiving fresh context values.

If your build system produces duplicates modules in the output (which can happen with symlinks), this can break context. Passing something via context only works if SomeContext that you use to provide context and SomeContext that you use to read it are exactly the same object, as determined by a === comparison.

Usage

Call useContext at the top level of your component to read and subscribe to context.

```
import { useContext } from 'react';

function Button() {
  const theme = useContext(ThemeContext);
  // ...
```

useContext returns the context value for the context you passed. To determine the context value, React searches the component tree and finds the closest context provider above for that particular context.

To pass context to a Button, wrap it or one of its parent components into the corresponding context provider:

```
function MyPage() {
  return (
    <ThemeContext.Provider value="dark">
      <Form />
    </ThemeContext.Provider>
  );
}

function Form() {
  // ... renders buttons inside ...
}
```

It doesn’t matter how many layers of components there are between the provider and the Button. When a Button anywhere inside of Form calls useContext(ThemeContext), it will receive "dark" as the value.

NOTE:
useContext() always looks for the closest provider above the component that calls it. It searches upwards and does not consider providers in the component from which you’re calling useContext().

Updating data passed via context

Often, you'll want the context to change over time. To update context, combine it with state. Declare a state variable in the parent component, and pass the current state down as the context value to the provider.

```
function MyPage() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext.Provider value={theme}>
      <Form />
      <Button onClick={() => {
        setTheme('light');
      }}>
        Switch to light theme
      </Button>
    </ThemeContext.Provider>
  );
}
```

useReducer

useReducer is a React Hook that lets you add a reducer to your component.

```
const [state, dispatch] = useReducer(reducer, initialArg, init?)
```

Call useReducer at the top level of your component to manage its state with a reducer.

```
import { useReducer } from 'react';

function reducer(state, action) {
  // ...
}

function MyComponent() {
  const [state, dispatch] = useReducer(reducer, { age: 42 });
  // ...
```

Parameters

- **reducer**: The reducer function that specifies how the state gets updated. It must be pure, should take the state and action as arguments, and should return the next state. State and action can be of any type.

- **initialArg**: The value from which the initial state is calculated. It can be a value of any type. How the initial state is calculated from it depends on the next init argument.

- **optional init**: The initializer function that should return the initial state. If it's not specified, the initial state is set to initialArg. Otherwise, the initial state is set to the result of calling init(initialArg).

Returns

useReducer returns an array with exactly two values:

The current state. During the first render, it's set to init(initialArg) or initialArg (if there's no init). The dispatch function that lets you update the state to a different value and trigger a re-render.

Caveats

useReducer is a Hook, so you can only call it at the top level of your component or your own Hooks. You can't call it inside loops or conditions. If you need that, extract a new component and move the state into it.

In Strict Mode, React will call your reducer and initializer twice in order to help you find accidental impurities. This is development-only behavior and does not affect production. If your reducer and initializer are pure (as they should be), this should not affect your logic.

The result from one of the calls is ignored.

Usage

Call useReducer at the top level of your component to manage state with a reducer.

```
import { useReducer } from 'react';

function reducer(state, action) {
  // ...
}

function MyComponent() {
  const [state, dispatch] = useReducer(reducer, { age: 42 });
  // ...
```

useReducer returns an array with exactly two items:

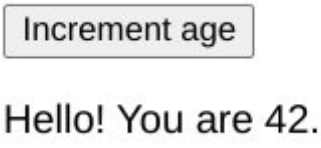
The current state of this state variable, initially set to the initial state you provided.
The dispatch function that lets you change it in response to interaction.

To update what's on the screen, call dispatch with an object representing what the user did, called an action:

```
function handleClick() {
  dispatch({ type: 'incremented_age' });
}
```

React will pass the current state and the action to your reducer function. Your reducer will calculate and return the next state. React will store that next state, render your component with it, and update the UI.

```
import { useReducer } from 'react';

function reducer(state, action) {
  if (action.type === 'incremented_age') {
    return {
      age: state.age + 1
    };
  }
  throw Error('Unknown action.');
}

export default function Counter() {
  const [state, dispatch] = useReducer(reducer, { age: 42 });

  return (
    <>
      <button onClick={() => {
        dispatch({ type: 'incremented_age' })
      }}>
        Increment age
      </button>
      <p>Hello! You are {state.age}.</p>
    </>
  );
}
```



Fetching data with useReduer

Using the useReducer() hook, we are going to fetch data from the API. We have performed data fetching in the previous article from **'https://reqres.in'** API using Axios library; here, we will be using useReducer() hook for fetching data.

Let's look at the example first with the use of useEffect() hook with the same example we used for useReducer() hook,

First, to start with, we need to install the Axios library using a command mentioned below.

//GetData_Reduce.js

```
import React, { useEffect, useReducer } from 'react'
import axios from 'axios'

const initialState = {
   user: {},
   loading: true,
   error: ''
}

const reduce = (state, action) => {
   switch (action.type) {
      case 'OnSuccess':
         return {
            loading: false,
            user: action.payload,
            error: ''
         }
      case 'OnFailure':
         return {
            loading: false,
            user: {},
            error: 'Something went wrong'
         }

      default:
         return state
   }
}

function GetData_Reduce() {
   const [state, dispatch] = useReducer(reduce, initialState)
```

```
  useEffect(() => {
    axios.get('https://reqres.in/api/users/2')
      .then(response => {
        dispatch({ type: 'OnSuccess', payload: response.data.data })
      })
      .catch(error => {
        dispatch({ type: 'OnFailure' })
      })
  }, [])

  return (
    <div>
      {state.loading ? 'Loading!! Please wait' : state.user.email}
      {state.error ? state.error : null}
    </div>
  )
}

export default GetData_Reduce
```

//App.js

```
import React from 'react';
import './App.css';
import GetData_Reduce from './components/GetData_Reduce;

function App() {

  return (
    <div className="App">
      <GetData_Reduce/>
    </div>
  );
}

export default App;
```

First, it will display the loading screen.
Once the data is loaded from the API, the data will be displayed on the screen.

useState vs useReduce

What is the difference between useState() and useReducer() hook?

Scenario	useState()	useReducer()
Type of state	Use it when working with Number, Boolean, and String	Use it when working with object or Array
Number of state Transition	useState hook has only one or two transitions so it should be used when you have only 1 or 2 setState calls	useReducer() hook should be used when many transitions so it must be used when we have many setState that need to be called
Related State Transition	No related transition	It includes state transition
Business Logic	useState has no business logic	When your application involves complex data transformation or manipulation that this should be used
Local vs Global	When dealing with a single component and need to perform operations locally than useState should be used	When you want to deal with multiple components for passing data from one component to another then useReducer() is a better approach.